DECLARATION

I hereby declare that the project entitled **"VidhanAI – AI-Powered Legal Awareness and Assistance Platform"** is a genuine work carried out by me in partial fulfilment of the requirements for the curriculum of the IV Semester, **Master of Computer Applications**, at **St. Agnes College (Autonomous), Mangaluru**.

This project has been completed under the valuable guidance of **Mrs. Panchajanyeswari H, Assistant Professor, PG Department of Computer Applications, St. Agnes College (Autonomous), Mangaluru**. Her continuous guidance, encouragement, and valuable suggestions greatly contributed to the successful completion of this project.

I further declare that the work presented in this project is original and has not been submitted by me, either wholly or partly, to any other University or Institution for the award of any degree, diploma, or any other academic qualification.

Brijesh **Place: Mangaluru**

2484092 **Date:**

Acknowledgement

I present my project entitled **"VidhanAI – AI-Powered Legal Awareness System"** with great pleasure and satisfaction. I sincerely thank everyone who supported and encouraged me throughout the successful completion of this project.

I express my heartfelt gratitude to our respected **Principal, Sr. Dr. M. Venissa A. C.,** and our **PG Coordinator, Sr. Dr. M. Vinora A. C.,** for their encouragement and support.

I am deeply grateful to **Mrs. Panchajanyeswari H.,** my project guide and **Head of the Department, PG Department of Computer Applications,** for her invaluable guidance, constant support, and encouragement, which greatly contributed to the successful completion of this project.

I also thank all the faculty members of the **Post Graduate Department of Computer Applications** for their support and guidance. Finally, I express my heartfelt gratitude to my parents, family, and friends for their constant encouragement and support throughout this journey.

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NUMBER
1.	**INTRODUCTION** 1.1 Introduction of the System 1.2 Overview 1.3 Background 1.4 Brief Note on Existing System 1.5 Objectives 1.6 Scope of the System 1.7 Structure of the System 1.8 End User	1-5
2.	**SOFTWARE REQUIREMENT SPECIFICATION (SRS)** 2.1 Introduction 2.2 Purpose 2.3 Scope 2.4 Overall Description 2.5 Special Requirements 2.6 Functional Requirements 2.7 Design Constraints 2.8 System Attributes 2.9 Other Requirements 2.10 Software / Hardware Requirements	6-13
3.	**SYSTEM DESIGN** 3.1 Introduction 3.2 Assumptions and Constraints 3.3 Functional Decomposition 3.4 System Software Architecture	14-22

	3.5 System Technical Architecture 3.6 System Hardware Architecture 3.7 External Interfaces 3.8 Description of Programs	
4.	**DATABASE DESIGN** 4.1 Introduction 4.2 Purpose and Scope 4.3 Database Identification 4.4 Data Dictionary 4.5 Physical Design 4.6 Entity Relationship Diagram 4.7 Database Administration	23-34
5.	**DETAILED DESIGN** 5.1 Introduction 5.2 Structure of the Software 5.3 Activity Diagrams 5.4 Use Case Diagram 5.5 Class Diagram 5.6 Sequence Diagrams 5.7 Module Descriptions	35-44
6.	**USER INTERFACE** 7.1 Login 7.2 Main Screen / Home Page 7.3 Menu 7.4 Data Store / Retrieval / Update 7.5 Validation 7.6 View 7.7 On-Screen Reports 7.8 Alerts 7.9 Error Message	45-54

	7.10 GestureFlow AI 7.11 Admin Console	
7.	**TESTING** 8.1 Introduction 8.2 Test Reports 8.3 Testing Criteria	55-58
8.	**CONCLUSION** 9.1 Future Scope	59-60
—	**ABBREVIATIONS AND ACRONYMS**	61
9.	**BIBLIOGRAPHY**	62

LIST OF FIGURES

CHAPTER NO	DESCRIPTION	PAGE NO
1	1.1— System structure	2
3	3.1 — Functional decomposition	15
3	3.2 — System software architecture	166
3	3.3 — System technical architecture	177
3	3.4 — System hardware architecture (deployment view)	188
3	3.5 — Context flow diagram (DFD level 0)	20
3	3.6 — Data flow diagram, level 1	21
3	3.7 — Data flow diagram, level 2 (Ask AI pipeline)	22
4	4.1 — Entity-Relationship diagram	30
5	5.1 — Structured chart, VidhanAI module overview (Admin · Frontend + GestureFlowAI · Backend)	34
5	5.2 — Activity diagram, Frontend Module (frontend/, incl. GestureFlowAI)	36
5	5.3 — Activity diagram, Backend & AI Services Module (backend/)	37
5	5.4 — Activity diagram, Admin Module (admin/)	38
5	5.5 — System use case diagram	40
5	5.6 — Class Diagram of the VidhanAI System	41
5	5.7 — Sequence diagram, admin announcement published & surfaced on frontend (Admin · Backend · Frontend)	42

CHAPTER 1
INTRODUCTION

1.1 Introduction of the System

India used the Indian Penal Code, 1860 (IPC) for more than 160 years. On 1 July 2024 it was replaced by the Bharatiya Nyaya Sanhita, 2023 (BNS), and almost every familiar section number changed with it. Cheating moved from "Section 420" to Section 318, murder from "Section 302" to Section 103, and hundreds of other provisions were merged, split, or renumbered. Lawyers keep up with such changes through commentaries and case law, but ordinary citizens usually cannot. This left a real awareness gap at the very moment the law behind everyday life was rewritten.

VidhanAI was built to close that gap. It is a full-stack web application that pairs a curated database of 358 BNS sections and 511 IPC sections with retrieval-augmented generation (RAG) on top of large language models. A user can type a plain question — "my phone was stolen in a market, what happens now?" — and get an answer grounded in the real statutory text, with the current BNS section cited first and its older IPC counterpart shown for context. The project grows a full learning platform around this core: side-by-side IPC↔BNS comparison, a quiz engine, a comic-story mode, a voice-enabled law tutor, and Free/Pro subscription plans with server-enforced gating and Razorpay billing.

1.4 Overview

The project is made up of three applications that work together:

- Citizen frontend (frontend/) — a single-page application at www.vidhanai.me, where every feature page loads as a lazy-loaded route.
- Admin console (admin/) — a separate front-end application with its own authenticated session, used for moderating reviews, managing law content, posting announcements, and viewing platform statistics.

- Backend (backend/) — a Python web service that holds all the business rules: authentication, the RAG pipeline, plan gating, subscription handling, and the admin API.

All data sits in a managed cloud document database across 13 collections, alongside a vector similarity index of 935 embeddings that covers every BNS and IPC section. Section 2.10 lists the full technology stack, including specific frameworks, hosting, and versions.

Figure 1.1 gives a first look at how these three applications and the surrounding third-party services fit together; Chapter 3 breaks this same picture down into its software, technical, and hardware views.

1.5 Background

The reason for building this is practical. During the switch to the new code, most material online still quotes IPC numbers, while the police and courts have already moved to the BNS. A tool that answers using both codes at once — and explains how they map to each other — is far more useful in this transition period than either code on its own.

This project was developed as an individual academic project. The motivation, scope, and every technical decision described in this report were driven by that transition problem rather than by an external client brief — VidhanAI was conceived, designed, and built end to end as a response to a real, dated event in Indian law.

1.6 Brief Note on Existing System

Before VidhanAI, a citizen trying to understand a legal situation had three realistic options, none of them satisfying. General-purpose search engines return news articles and law-firm blog posts that summarise a section loosely and rarely distinguish whether they are talking about the old IPC or the new BNS. General-purpose chatbots can explain law in plain language, but they are not grounded in any specific statutory dataset — they can and do invent section numbers, which is a serious problem in a legal context. And official government portals such as India Code publish the authoritative text but expect the reader to already know which section they are looking for; they offer no scenario search and no bridge between the old and new codes.

None of these existing options solve the actual problem of the transition period: knowing which BNS section now covers a situation a citizen used to know by its IPC number, and getting an answer that is both grounded in real statutory text and explained simply. That gap is exactly what VidhanAI's retrieval-augmented design and side-by-side IPC↔BNS comparison are built to close.

1.7 Objectives

- Answer legal questions in plain language, grounded in the real text of BNS 2023 and IPC 1860, and cite the correct sections (BNS first, with IPC given as historical context).
- Never make up a section: retrieval picks the sections from the project's own dataset, and the language model only explains them.
- Make the IPC→BNS renumbering easy to see and search, through the compare view.
- Teach as well as answer, using quizzes, comic stories, a voice tutor, and a learning hub meant for students and first-time readers of law.
- Offer seven Indian languages to Pro users while keeping the Free plan genuinely usable (English, five questions a day).
- Run a complete commercial flow end to end: Razorpay subscriptions (UPI Autopay and cards), server-side plan enforcement, cancellation at cycle end, and invoices.
- Stay operable by a non-programmer through the admin console — moderation, law CMS, announcements, and statistics.
- Make the site usable without a mouse or touchscreen: GestureFlow AI reads webcam hand gestures for scrolling and clicking, open to every visitor regardless of plan or login state.

1.8 Scope of the System

In scope: citizen Q&A over BNS/IPC using RAG; law search, section detail, and comparison; quiz, comic, and tutor learning modes; user accounts with email-OTP verification and Google OAuth; Free and Pro plans with server-side gating; Razorpay test-mode subscription billing, including cancellation; reviews with admin moderation; a contact form with email delivery; touchless webcam gesture control of the site; and the admin console.

Out of scope: legal advice (the system is only an awareness tool and says so clearly); live payments (the code deliberately rejects non-test Razorpay keys until KYC is done); civil, tax, and procedural codes beyond BNS/BNSS/IPC; and native mobile apps.

1.9 Structure of the System

The repository is a three-application monorepo that shares one database. Figure 1.2 shows each application and, for the backend, the packages inside it.

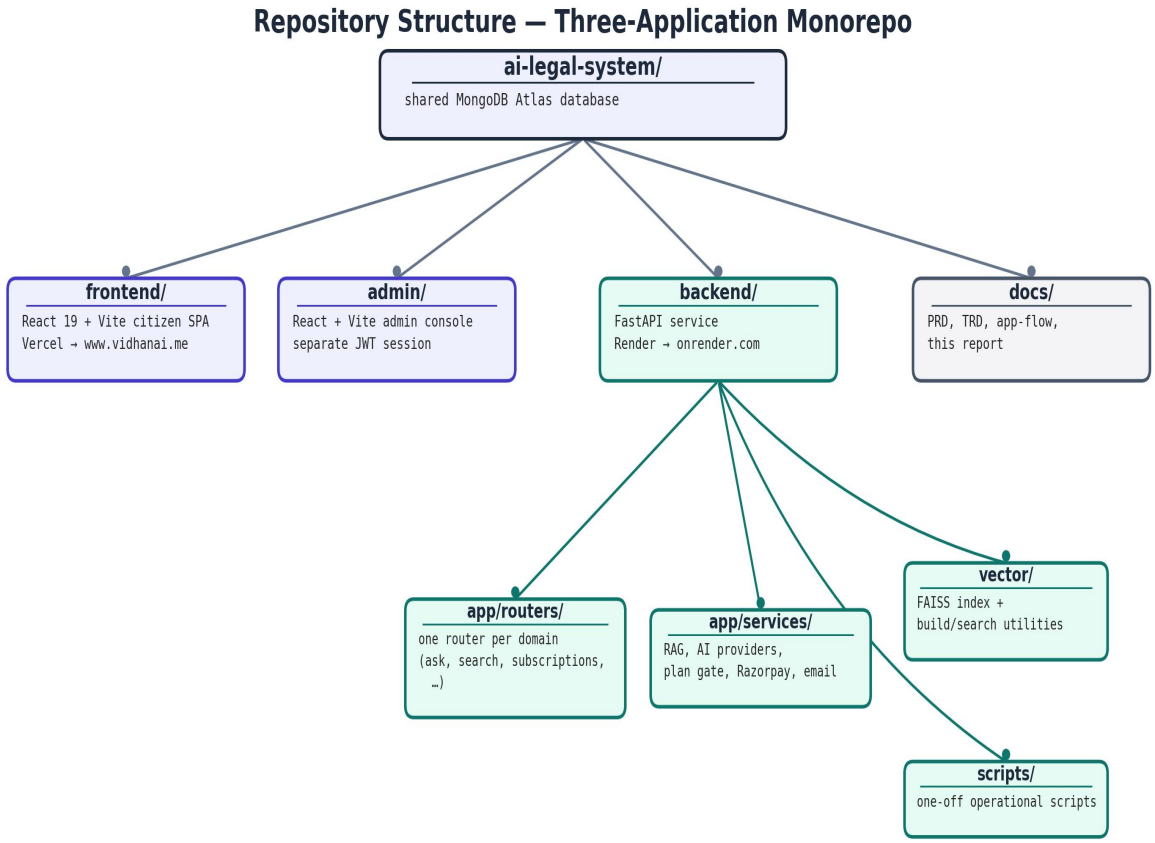


Figure 1.1 — System structure

1.10 End User

User class	Description	Typical actions
Anonymous visitor	Not signed in; browsing the public site	Home-page demo chat (2 questions/day per IP), reading public pages, pricing
Free user	Registered, email-verified account	5 AI questions/day (English), law search & compare, Know-Your-Rights
Pro subscriber	Active Razorpay subscription; plan_status = “pro”	Unlimited questions, 7 languages, voice input, tutor with TTS, quiz hub, comic mode
Administrator	Separate admin account (own JWT)	Dashboard overview, manage users, moderate & feature reviews, law CMS, browse AI query logs, post announcements, manage admin accounts

GestureFlow AI is available to every row in this table equally — it runs client-side with no account or plan check, so an anonymous visitor gets the same touchless control as a Pro subscriber.

CHAPTER 2

Software Requirement Specification (SRS)

2.1 Introduction

This SRS sets out what VidhanAI has to do, who it is for, and the constraints it works under. It describes the system as it was actually built and deployed — every requirement below exists in the repository and can be checked against the live site — and the functional requirements table links each item to the API route that fulfils it.

2.2 Purpose

The purpose of this document is to record, in one place, the functional behaviour, non-functional attributes, and constraints that the finished VidhanAI application satisfies, so that the design chapters that follow can be read against a single, agreed baseline rather than against scattered notes.

2.3 Scope

The SRS covers the whole product surface described in section 1.8 — citizen Q&A, law search and comparison, the learning suite, accounts, commerce, community features, and administration — for the three applications in the monorepo (citizen frontend, admin console, backend). It does not cover infrastructure procurement or the third-party services themselves (Razorpay, Groq, Gemini, Resend, Google), only VidhanAI's use of them.

2.4 Overall Description

2.4.1 Product Perspective

VidhanAI is a self-contained product made of replaceable parts. The browser apps are stateless clients, the FastAPI service is the single authority for business rules, MongoDB holds the documents, and FAISS handles similarity search. The LLM providers sit behind one service layer and can be swapped freely (Groq is primary for English, Gemini handles

Indic scripts and acts as fallback). Payments and email are handed off to Razorpay and Resend, and the system never stores any card or bank details. One part of the citizen frontend is deliberately outside this client-server picture: GestureFlow AI, a touchless hand-gesture controller, runs entirely inside the browser with no backend or account dependency at all — it is a single script loaded once, and it works identically whether the visitor is signed in or anonymous.

2.4.2 Product Functions

- Conversational legal Q&A with streaming answers, voice input for Pro users, and per-message actions such as copy, simplify, and history bookmark.
- A law explorer with keyword search, section detail pages, autosuggest, and a law-of-the-day.
- IPC ↔ BNS comparison, showing the structural mapping together with an AI-generated summary.
- A learning suite made up of a quiz engine, a comic-story generator, and a law tutor with JD voice lessons and TTS.
- GestureFlow AI — touchless, webcam-driven hand-gesture control of the whole site (point to scroll or move a cursor, pinch to click), available to every visitor regardless of plan.
- Accounts and identity: signup with email OTP, login, Google OAuth, and a profile with avatar and phone.
- Commerce: Free and Pro plans, Razorpay subscription checkout, activation by webhook or API reconciliation, cancellation at cycle end, and invoice retrieval.
- Community and support: user reviews (which admins can feature on the home page) and a contact form that sends an acknowledgement email.
- Administration console: an overview dashboard, user management, review moderation, a law CMS for BNS/IPC content, an AI query-log browser, a platform announcements composer, and admin account/password management — all behind a separate admin login.

2.4.3 User Characteristics

The main audience is non-lawyers — students, working people, and first-time internet users. Because of this, the interface stays away from legal jargon, can explain a section in class-8-level language on request, supports Indian languages, and always shows the exact sections behind each AI answer. Administrators are treated as non-programmers, so every task they need is available in the console.

2.4.4 General Constraints

- The Free plan has to stay usable without paying: 5 questions a day, English only, and all law-reading features open.
- AI answers must be grounded — nothing is generated without retrieved statutory context.
- The free-tier backend has only 512 MB of RAM, which cannot hold the ~400 MB embedding model, so semantic search falls back cleanly to keyword retrieval when it is switched off.
- Payments run in Razorpay TEST mode only; the client refuses live keys on purpose.
- Every string a user submits is sanitised — HTML stripped and length capped — before it reaches the database or an LLM.

2.4.5 Assumptions

- MongoDB Atlas and the LLM providers are assumed reachable, and each integration degrades gracefully (keyword search, provider fallback, cached data) instead of returning a bare 500.
- Razorpay is assumed to deliver each webhook at least once. Since delivery timing is not guaranteed, the system also reconciles subscription state directly against the Razorpay API.
- The statutory dataset (bns.json / ipc.json) is trusted content curated by the project.

2.5 Special Requirements

- Legal honesty: a disclaimer stays on screen at all times, reminding users that VidhanAI is an awareness tool and not a replacement for professional legal advice.

- Grounding guarantee: the LLM only explains sections chosen by retrieval from the project dataset, and if nothing relevant turns up the system says so rather than guessing.
- Payment integrity: Pro access is granted only on the server — either through a signature-verified Razorpay webhook or by fetching the subscription from Razorpay's API with the secret key. Nothing the browser sends can hand out a plan.
- A single phone number cannot hold two active Pro subscriptions; this is checked at subscribe time.

2.6 Functional Requirements

ID	Requirement	Implemented By
FR-01	User registration with OTP verification	Signup & OTP APIs
FR-02	User login with JWT authentication	Login API
FR-03	Google account login	Google OAuth API
FR-04	Update profile and phone number	Profile APIs
FR-05	Ask AI legal questions	Ask AI APIs
FR-06	Daily usage limit by plan	Plan quota service
FR-07	Pro-only multilingual support	Plan access control
FR-08	Search laws with suggestions	Law search APIs
FR-09	Compare IPC and BNS sections	Comparison APIs
FR-10	Generate and submit quizzes	Quiz APIs
FR-11	Generate comic-style	Comic Story API

	explanations	
FR-12	Voice-based legal tutor	Tutor & TTS APIs
FR-13	Create subscription	Subscription API
FR-14	Activate Pro subscription	Webhook & Reconcile APIs
FR-15	Cancel subscription at period end	Cancellation API
FR-16	View subscription status	Plan Status API
FR-17	Submit and manage reviews	Review APIs
FR-18	Send contact request	Contact API
FR-19	Admin dashboard statistics	Admin Stats API
FR-20	Manage user accounts	User Management APIs
FR-21	Moderate reviews	Review Admin APIs
FR-22	Manage legal content	Law CMS APIs
FR-23	View AI query logs	Query Log API
FR-24	Manage announcements	Announcement APIs
FR-25	Manage admin accounts	Admin Account APIs
FR-26	Hand gesture website control	Gesture Control Module

2.7 Design Constraints

- PyMongo is synchronous (there is no async driver), so the FastAPI handlers are plain def functions.
- Groq's JSON mode returns only objects, so structured prompts ask for a {"steps": […]} wrapper instead of a bare array.

- The FAISS index and its id-mapping have to be rebuilt whenever the law collections are re-seeded, because the ids embed MongoDB ObjectIds.
- The React apps use plain co-located CSS — no Tailwind or CSS Modules — with lazy-loaded routes.
- Every rate-limited endpoint takes request: Request as its first parameter, which SlowAPI requires.

2.8 System Attributes

Attribute	**Provision in VidhanAI**
Security	BCrypt hashing, JWT, secure webhooks, input validation, CORS.
Reliability	AI fallback, keyword search fallback, error recovery.
Availability	Cloud deployment, restartable backend, responsive frontend.
Usability	Simple UI, multilingual support, voice features, mobile-friendly.
Maintainability	Modular design, separate services, organized code.
Portability	Container-ready, cloud compatible, environment configuration.

2.9 Other Requirements

Regulatory: the platform keeps no payment-instrument data, so the PCI burden stays with Razorpay, and it shows test-mode notices anywhere simulated money appears. Data retention: chat history is kept per account and can be deleted by an admin on request. Licensing: the statutory text is public-domain Indian law, and the third-party services are used under their own commercial terms.

2.10 Software / Hardware Requirements

2.10.1 Used for Development

Layer	Technology
Frontend	React, Vite, React Router, Three.js, CSS
Backend	Python, FastAPI, Uvicorn, Pydantic
Database	MongoDB Atlas
AI / RAG	Groq, Gemini, FAISS, Sentence Transformers
Payments / Email	Razorpay, Resend
Authentication	JWT, BCrypt, Google OAuth, Email OTP
Development Tools	Git, GitHub, VS Code, Vercel, Render
Development Machine	Windows 11, 8 GB+ RAM

2.10.2 Required for Implementation

Component	Minimum Requirement
Backend Host	Linux server, 512 MB RAM (2 GB+ for FAISS).
Frontend Host	Static hosting (Vercel) or any Vite-supported server.
Database	MongoDB 6+ or MongoDB Atlas.
Client	Modern browser with JavaScript and internet connection.
Third-Party Services	Razorpay, Groq, Gemini, Resend, Google

	OAuth.

CHAPTER 3

System Design

3.1 Introduction

This chapter explains how the requirements from Chapter 2 are split into processes and data flows. The design uses a familiar layered structure — presentation (the two SPAs), application logic (the FastAPI routers and services), and data (MongoDB and FAISS) — with third-party systems attached at the edges through verified, replaceable interfaces.

3.2 Assumptions and Constraints

- One backend instance serves all traffic. Scaling horizontally would only need a shared MongoDB and a FAISS artefact rebuilt per node, since the index is read-only at runtime.
- The LLM keeps no state between requests; everything that must persist (history, tutor session state) is stored server-side in MongoDB.
- Payment state really belongs to Razorpay, and the local records are just a synchronised cache — which is exactly why reconciling against their API is always an option.

3.3 Functional Decomposition

Figure 3.1 breaks the system into five functional pillars. Each pillar maps to a group of backend routers and lazy-loaded frontend routes, so any one of them can be changed or removed without disturbing the rest.

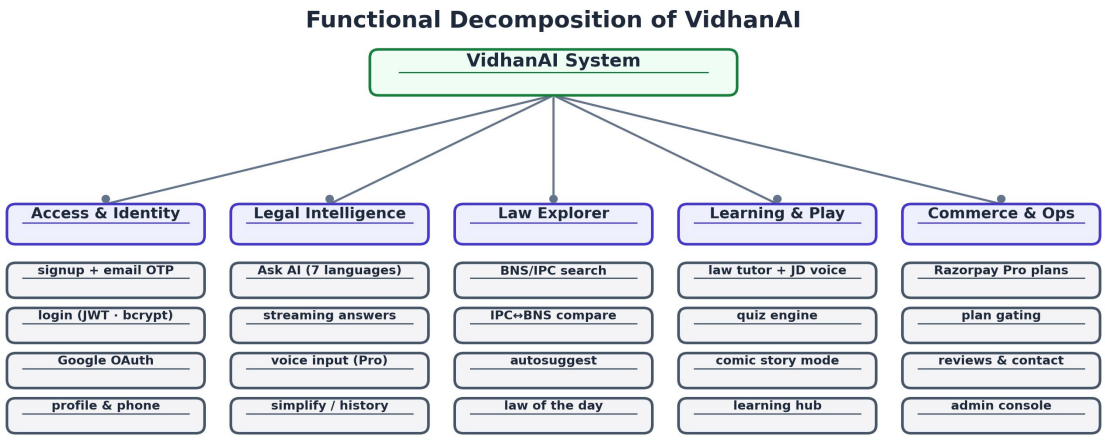


Figure 3.1 — Functional decomposition

3.4 System Software Architecture

Figure 3.2 shows VidhanAI as three software layers. Presentation covers the two React single-page applications; application logic is every FastAPI router and service; and the data layer is MongoDB Atlas together with the FAISS vector index. Each layer only talks to the one directly below it, which is what lets, for instance, the retrieval implementation change from FAISS to another vector store without touching a single router.

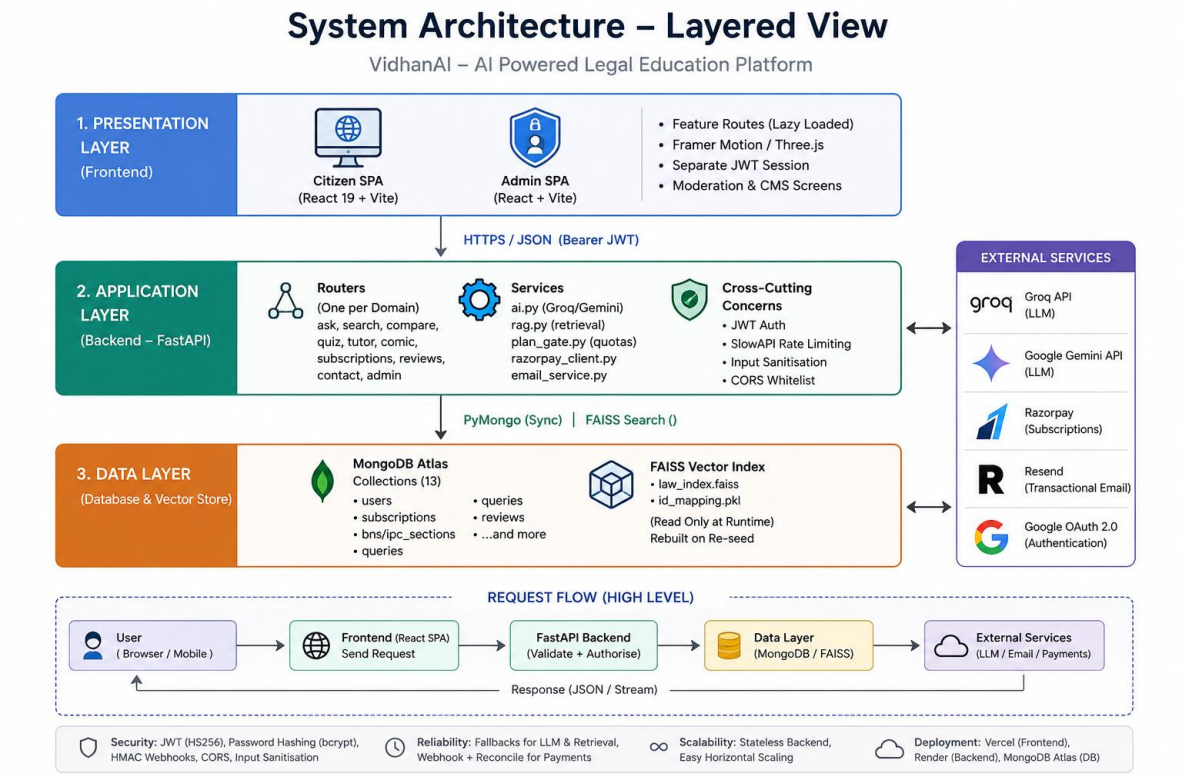


Figure 3.2 — System software architecture

3.5 System Technical Architecture

Figure 3.3 names the concrete technology and protocol at each tier. The client tier is a static Vite bundle talking HTTPS/JSON with SSE for streaming; the service tier is FastAPI on Uvicorn with JWT/OAuth2 authentication and SlowAPI rate limiting; the AI/retrieval tier wraps the embedding model, FAISS index, and the two LLM SDKs; and the integration tier covers the Razorpay, Resend, Google, and MongoDB wire protocols. Nothing in the diagram is aspirational — every item is a dependency already pinned in the project's requirements files.

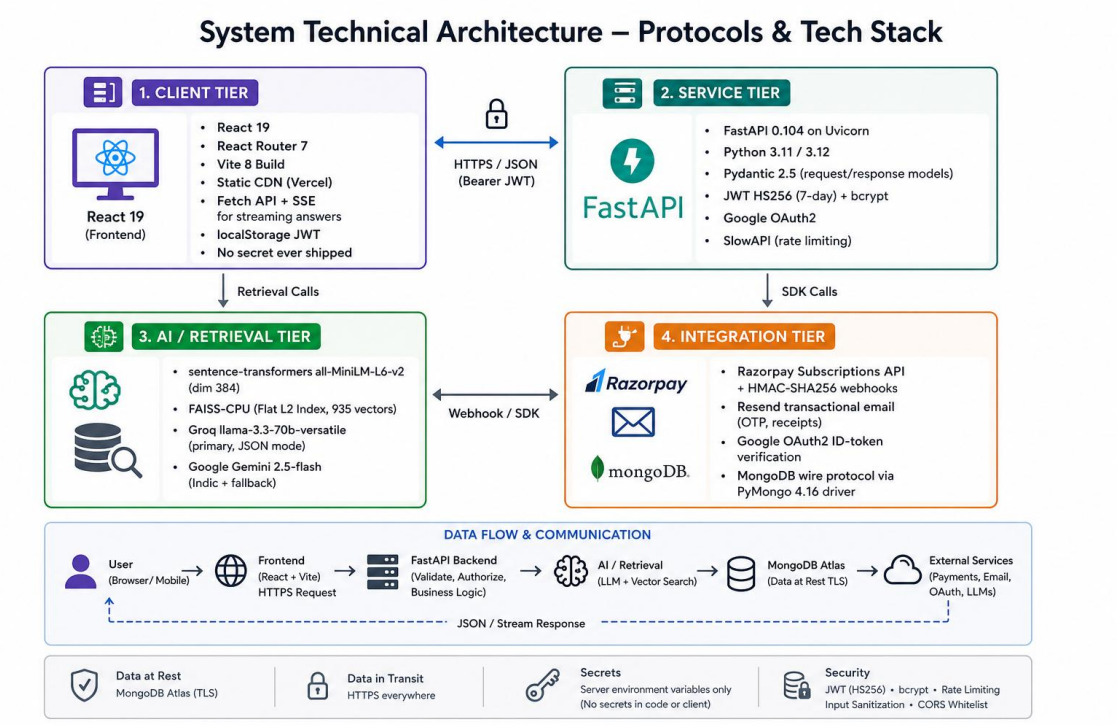


Figure 3.3 — System technical architecture

3.6 System Hardware Architecture

Figure 3.4 shows where each part physically runs. The browser talks to Vercel's CDN for static assets and to a single Render web-service instance for the API. That instance is the only component with access to the MongoDB Atlas cluster and to the outbound SaaS providers — the browser never talks to Razorpay, Groq, Gemini, or Resend directly, and no API key ever reaches it.

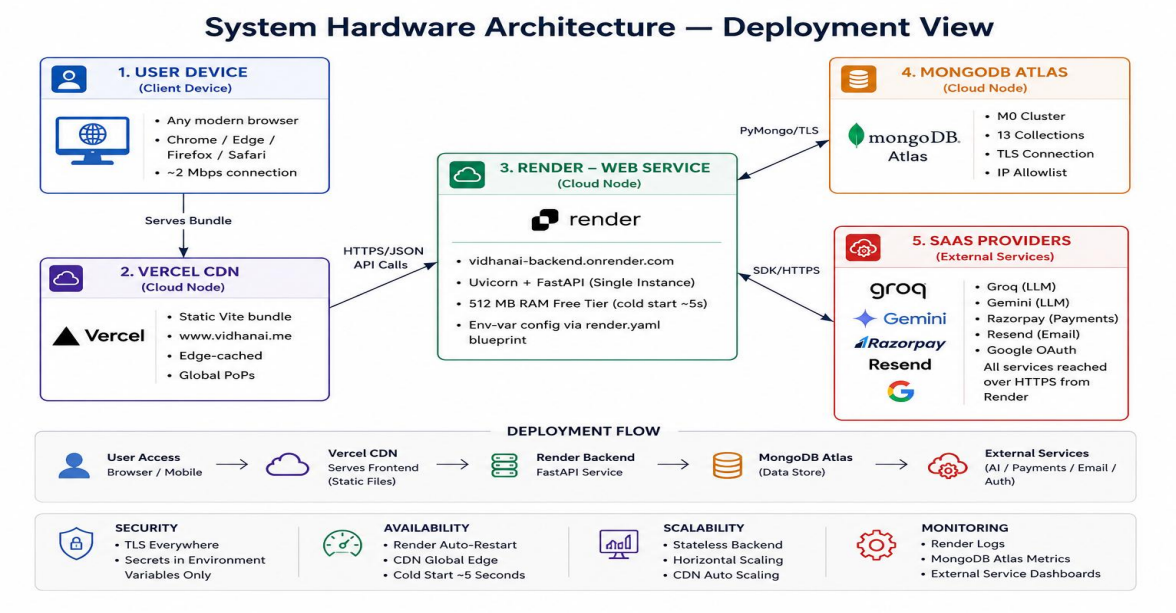


Figure 3.4 — System hardware architecture (deployment view)

3.7 External Interfaces

Interface	Direction	Protocol / Notes
Razorpay Subscriptions API	Backend → Razorpay	HTTPS REST; create/fetch/cancel a subscription; test-mode keys only
Razorpay webhooks	Razorpay → Backend	HTTPS POST, HMAC-SHA256 signed (X-Razorpay-Signature)
Google OAuth 2.0	Browser → Google → Backend	ID-token issued to the browser, verified server-side before a JWT is minted
Groq / Gemini SDKs	Backend → provider	HTTPS REST via official SDKs; Groq primary, Gemini for Indic languages and fallback
Resend	Backend → Resend	HTTPS REST; OTP mail, contact acknowledgement, receipts
MongoDB Atlas	Backend → database	MongoDB wire protocol over TLS via PyMongo

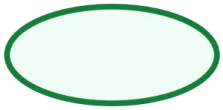
3.8 Description of Programs

3.8.1 Context Flow Diagram (CFD)

At the context level (Figure 3.5), the system exchanges data with five outside entities. Citizens and administrators work through the browser; Razorpay both receives API calls (create, cancel, fetch a subscription) and sends signed webhooks back; Google verifies OAuth ID tokens; and Resend delivers OTPs, receipts, acknowledgements, and contact mail.

Basic DFD Notation:

The symbols used in the diagrams above are shown in the table below:

Name	Notation	Description
Process		A process transforms incoming data flow into outgoing data flow. The processes are shown by named circles.
Datastore		Data stores are repositories of data in the system. They are sometimes also referred to as files.
Dataflows		Data flows are pipelines through which packets of information flow. Label the arrows with the name of the data that moves through it.
External Entity		External entities are objects outside the system with which the system communicates. External Entities are sources and destinations of the system's inputs and outputs.

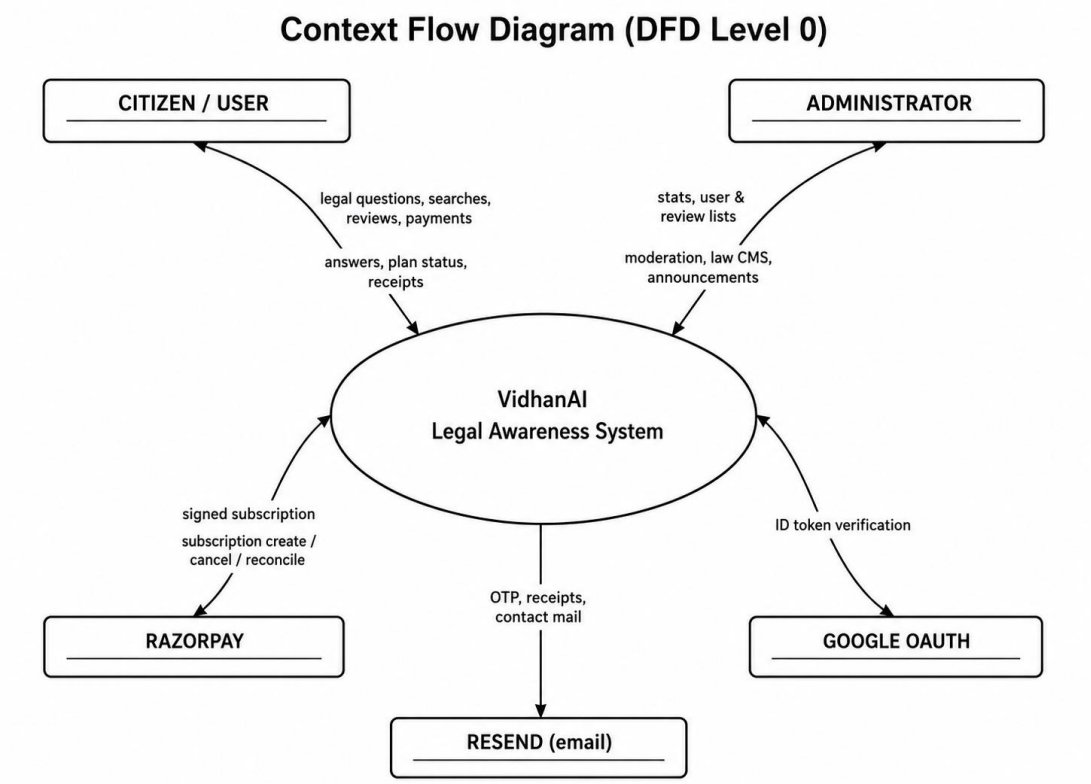


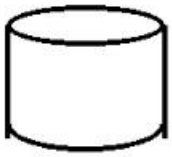
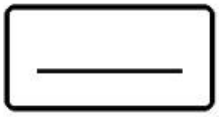
Figure 3.5 — Context flow diagram (DFD level 0)

3.8.2 Data Flow Diagrams (Levels 1 and 2)

Level 1 (Figure 3.6) shows six processes over five main data stores. The plan gate (4.0) sits deliberately between authentication and Ask AI: every request that counts against a quota passes through it, and it reads the same users store that the subscription process writes to — which is why Pro access takes effect the instant it is granted.

Basic DFD Notations:

Name	Notation	Description
Process		**Transforms input data into output.**

Data Store		**Stores and retrieves system data.**
Data Flow		**Represents movement of data.**
External Entity		**Represents a user or external system.**

3.8.2.1DFD Level 1:

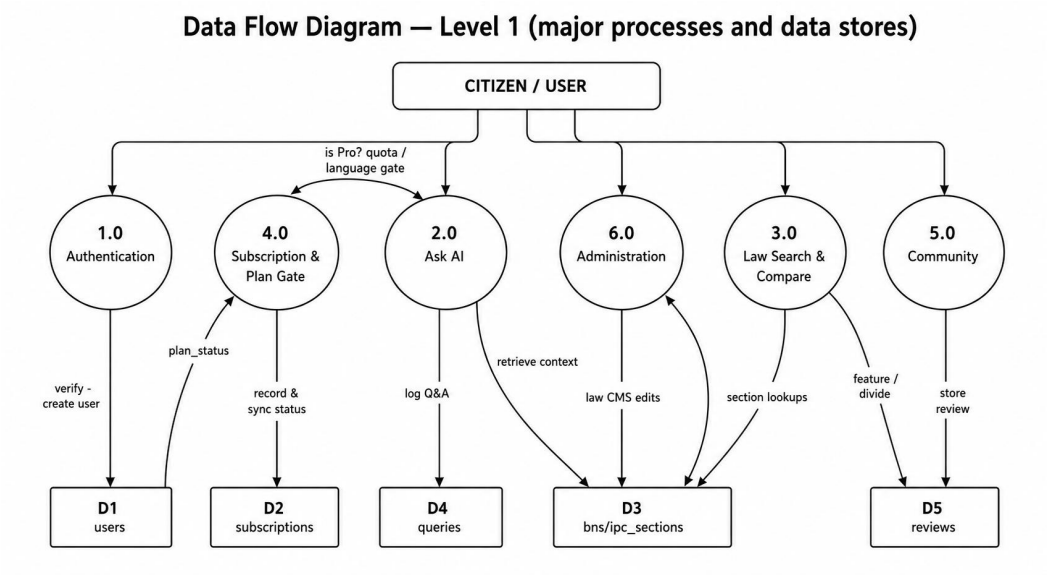


Figure 3.6 — Data flow diagram, level 1

Level 2 (Figure 3.7) expands Ask AI, the busiest process. The input is sanitised, the plan gate counts the question (returning HTTP 429 once a Free user hits the limit), and FAISS pulls the top-3 relevant sections. Only when the vector index is unavailable does the MongoDB keyword fallback run, because an explicit is_available() check separates “nothing matched” from “index switched off”. Groq then writes the grounded answer (Gemini takes over for

Indic languages or on failure), the reply streams token by token, and the whole exchange is saved to history.

3.8.2.2DFD Level 2:

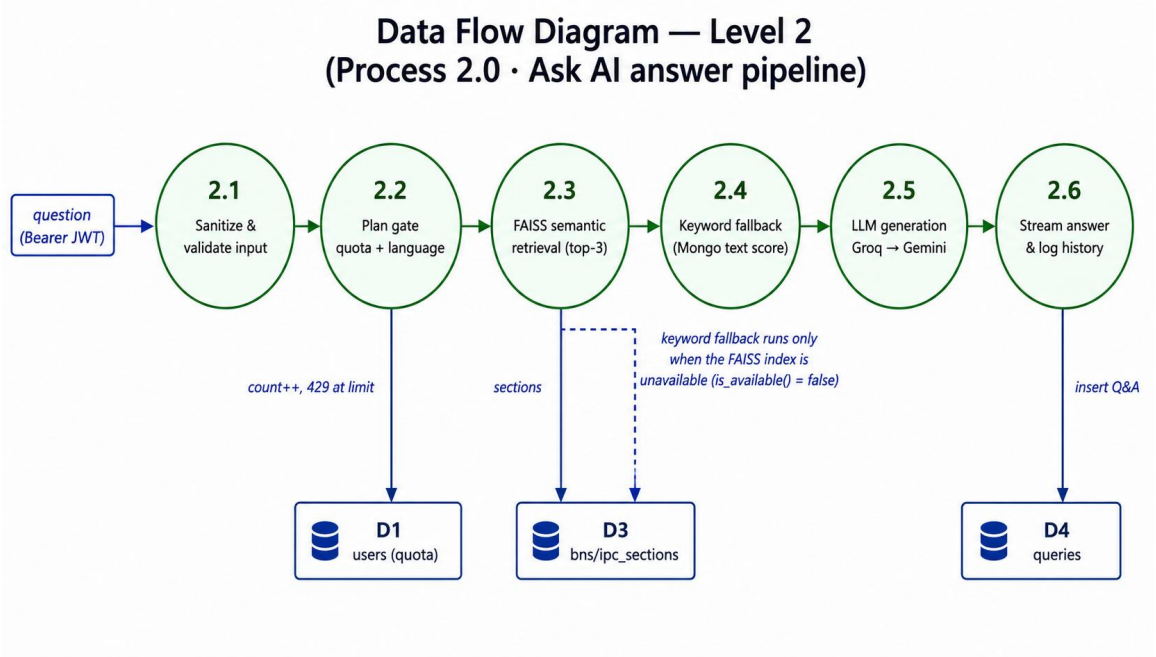


Figure 3.7 — Data flow diagram, level 2 (Ask AI pipeline)

CHAPTER 4

Database Design

4.1 Introduction

VidhanAI runs on MongoDB, a document database, rather than a relational one. The main data shapes here — statutory sections with nested subsections, punishments that may be plain strings or structured objects, and AI exchanges of varying length — fit documents well and would otherwise spread across many join tables in SQL. The relationships that do matter (subscription → user, review → user) are stored as indexed, foreign-key-style fields and enforced in the service layer.

4.2 Purpose and Scope

The database does four jobs: it holds (1) the statutory corpus that grounds every AI answer, (2) user identity and plan state, (3) commercial records kept in sync with Razorpay, and (4) operational content such as reviews, announcements, and settings. Thirteen collections cover all of this.

4.3 Database Identification

The table below identifies all 13 collections and the role each one plays. The seven marked "core" are defined field by field in section 4.4; the rest are summarised there as supporting collections.

#	**Collection**	**Role**
1	users	Core — identity, credentials, plan state
2	subscriptions	Core — Razorpay subscription lifecycle

3	reviews	Core — testimonials, admin-moderated
4	queries	Core — Ask AI chat history
5	bns_sections	Core — BNS 2023 statutory corpus (358 docs)
6	ipc_sections	Core — IPC 1860 statutory corpus (511 docs)
7	usage_quota	Core — per-IP anonymous demo counter
8	announcements	Supporting — admin "what's new" posts
9	newsletter_subscribers	Supporting — footer newsletter e-mails
10	admin_users	Supporting — administrator credential space
11	settings	Supporting — platform flags (e.g. maintenance mode)
12	comics	Supporting — generated comic-story cache
13	leaderboard	Supporting — game XP standings

4.4 Data Dictionary

Table 4.1 — users

Field	Type	Description
_id	ObjectId	Primary key
email	string (unique)	Login identity; the foreign key other collections reference
name / password	string / bcrypt hash	Display name; hash only for password accounts

verified	bool	Set after e-mail OTP confirmation
auth_provider	string	“password” or “google”
phone	string (E.164)	Required before subscribing; prefills Razorpay checkout
plan_status	“free” | “pro”	THE gate — read by every quota/feature check
qa_date / qa_count	string / int	Free-plan daily question counter (resets at IST midnight)
current_period_end	datetime	End of the paid period (shown as renewal/access date)
cancel_at_cycle_end	bool	True once the user cancels; access continues to period end
razorpay_subscription_id	string	The subscription this account’s Pro state derives from

Table 4.2 — subscriptions

Field	**Type**	**Description**
_id	ObjectId	Primary key
razorpay_subscription_id	string (unique)	Razorpay’s id (sub_…); upsert key
user_email	string → users	Owning account
plan_id / period	string / “monthly”|“annual”	Razorpay plan and billing period
status	string	created → active → halted/cancelled lifecycle mirror
current_period_end	datetime	From Razorpay current_end timestamp
cancel_at_cycle_end	bool	Cancellation scheduled (no refund model)
last_payment_id / last_invoice_id	string	Latest charge artefacts for support

is_test_mode	bool	Always true until live keys are enabled

Table 4.3 — reviews

Field	Type	Description
_id	ObjectId	Primary key
email / name / role	string	Author identity and self-described role
rating	int 1..5	Star rating
text	string	Review body (sanitised)
featured	bool	Set by admins; only featured reviews appear on the home page
created_at	datetime	Submission time (newest first)

Table 4.4 — queries (chat history)

Field	Type	Description
_id	ObjectId	Primary key
user_email	string → users	Asking account (or anonymous marker)
question / answer	string	The exchange as shown to the user
language	string	Answer language
bookmarked	bool	User bookmark toggle
created_at	datetime	For history ordering and admin analytics

Table 4.5 — bns_sections (358 documents)

Field	Type	Description
_id	ObjectId	Primary key
section_number	string (unique)	e.g. “303”, “85”
title / chapter	string	Statutory heading and chapter
description	string	Full section text
punishment	string | object	May be structured {imprisonment, fine, …}
keywords	[string]	Curated retrieval helpers
ai_summary	string	Pre-generated plain-language summary
subsections / illustrations / exceptions	nested	Statutory structure preserved as-is
is_punishable	bool	Distinguishes offence sections from definitions

Table 4.6 — ipc_sections (511 documents)

Field	Type	Description
_id	ObjectId	Primary key
section_number	string (unique)	e.g. “420”, “498A”
title / chapter / section_text	string	Heading, chapter, full text
bailable / cognizable / compoundable	bool/string	Procedural attributes
offence_category / max_punishment	string	Classification and ceiling
simple_explanation / meaning / ai_summary	string	Layered plain-language explanations
related_sections	[string]	Cross-references incl. BNS

		equivalents

Table 4.7 — usage_quota

Field	Type	Description
kind + ip + date	compound key	"anon_ask" per client IP per day
count	int	Questions consumed today (limit 2 for anonymous demo)

Table 4.8 — supporting collections

Collection	Purpose
announcements	Admin "what's new" posts targeted to all/free/pro audiences
newsletter_subscribers	Footer newsletter e-mails (announcements are mailed to them)
admin_users	Administrator accounts — separate credential space and JWT from citizens
settings	Platform flags (e.g. maintenance mode) toggled from the console
comics	Cache of generated comic stories keyed by topic
leaderboard	Game XP standings

4.5 Physical Design

Physical design covers how the logical schema above is actually indexed and queried. Three kinds of index are in use. Single-field indexes on section_number (both bns_sections and ipc_sections) and on chapter and offence_category back the section-detail and browse-by-chapter lookups with an O(log n) query instead of a collection scan. A compound text index over {title, description/section_text, keywords} on both law collections is what powers the keyword-search fallback the Ask AI pipeline drops into whenever the FAISS vector index is

unavailable. And email is kept unique on users, since it is both the login identity and the field every other collection joins against.

Outside MongoDB, the FAISS side of the corpus is a separate physical artefact: a flat L2 index of 935 vectors (384 dimensions each, from all-MiniLM-L6-v2) stored as law_index.faiss, with a parallel id_mapping.pkl that maps each vector position back to a MongoDB ObjectId and its source collection. The two files are read-only at runtime and must be rebuilt together — a stale mapping after a re-seed is exactly the failure mode described in section 5.7.

4.6 Entity Relationship Diagram

Figure 4.1 shows the entities and how they relate. In terms of cardinality, one user can have many subscriptions over time (with exactly one of them driving plan_status), along with many reviews and many history entries. Each BNS section maps to at most one IPC predecessor (1 : 0..1) through the ipc_section cross-reference, which reflects the sections that are brand new in BNS 2023. The supporting collections in the lower band are keyed independently and hold no foreign key into users.

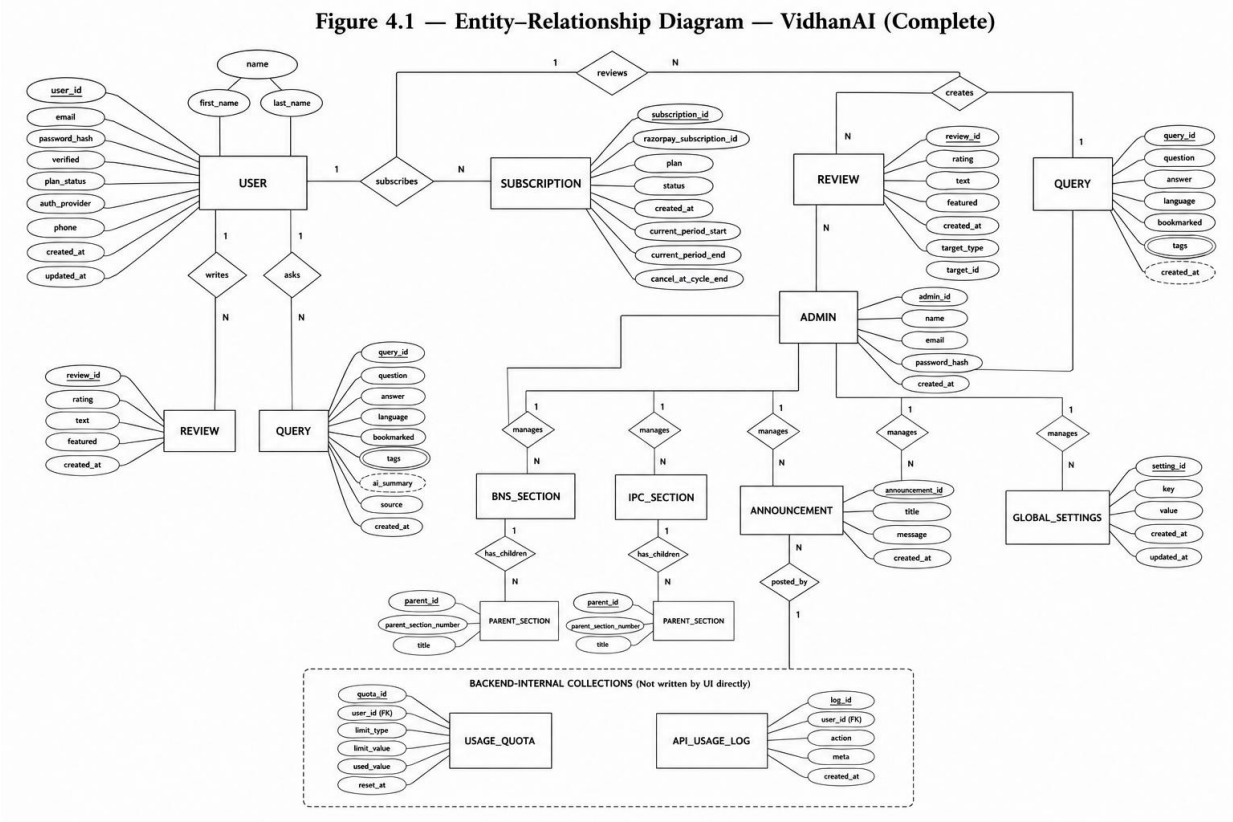
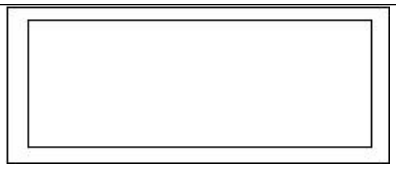
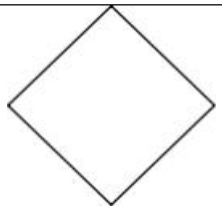
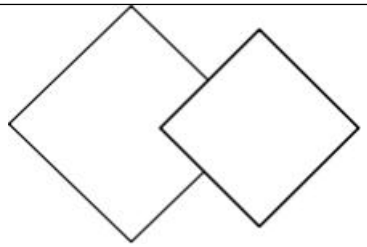
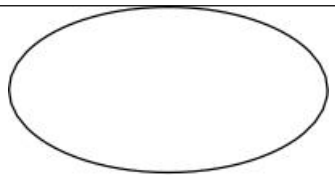
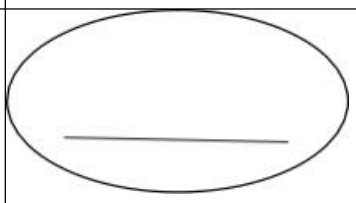
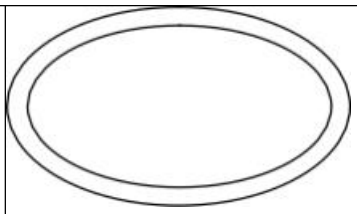
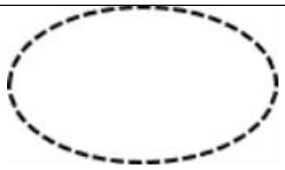
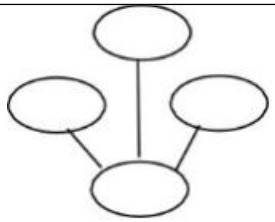


Figure 4.1 — Entity-Relationship diagram

ER Diagram Symbols and Notations:

1.Strong Entity: Strong entities are independent from other entities, and are called parent entities, since they may have weak entities that depend on them. They also have a primary key, distinguishing each occurrence of the entity**.**	

2. Weak Entity Weak entities depend on some other entity type. They don't have primary keys, and have no meaning in the ER diagram without their parent entity	
3. Relationship: Relationships are associations between or among entities	
4. Weak Relationship: Weak Relationships are connections between a weak entity and its owner.	
5. Attribute: Attributes are characteristics or properties of an entity.	
6. Key Attribute: A Key attribute is the attribute which uniquely identifies each entity in the entity set.	

7. Multivalued Attribute: Multivalued attributes are those that are can take more than one value.	
8. Derived Attribute: Derived attributes are attributes whose value can be calculated from related attribute values.	
9. Composite Attribute: A Composite attribute is composed of many other attributes.	

4.7 Database Administration

The legal datasets are managed through an automated database seeding process rather than manual data entry. The seed_bns.py and seed_ipc.py scripts read the curated bns.json and ipc.json files and populate the bns_sections and ipc_sections collections. During this process, important fields such as **section number**, **chapter**, and **law text** are indexed to enable faster searching and retrieval, as described in Section **4.5**. For backward compatibility with earlier

versions of the application, the seed_db.py script also populates the legacy laws_collection used by the initial law comparison module.

Whenever the legal dataset is updated or re-seeded, the existing FAISS vector index becomes outdated. To keep semantic search accurate, the vector/build_index.py script is executed immediately after the seeding process. This script generates fresh vector embeddings for every law section using the **all-MiniLM-L6-v2** Sentence Transformer model and creates new law_index.faiss and id_mapping.pkl files. These files are then used by the Retrieval-Augmented Generation (RAG) system to perform efficient semantic searches.

To simplify administration, the system provides a dedicated **Rebuild FAISS Index** option in the admin dashboard. This feature internally calls the POST /admin/rebuild-faiss API, allowing administrators to regenerate the vector index without using command-line tools or writing any code.

For security, administrator accounts are stored separately in the admin_users collection and use an independent authentication mechanism based on JWT (JSON Web Token). This separation ensures that even if a regular user account is compromised, it cannot access administrative functions such as content management, review moderation, or FAISS index management, thereby maintaining the security and integrity of the system.

CHAPTER 5

Detailed Design

5.1 Introduction

This chapter moves from processes down to individual modules and their internal logic. The backend is built as thin routers sitting over focused services. The two modules detailed in section 5.2 were picked because, together, they touch every architectural mechanism in the system: retrieval, generation, gating, external synchronisation, and payment integrity. The sections that follow — activity, use case, class, and sequence diagrams — describe the same two modules from complementary UML viewpoints.

5.2 Structure of the Software

5.2.1 Structured Charts

Figure 5.1 steps back from backend internals to the package as a whole: the three top-level modules — the admin console (admin/), the citizen frontend (frontend/) with GestureFlowAI as an in-browser sub-module that never calls the backend, and the backend & AI services layer (backend/) that both of the others reach over REST.

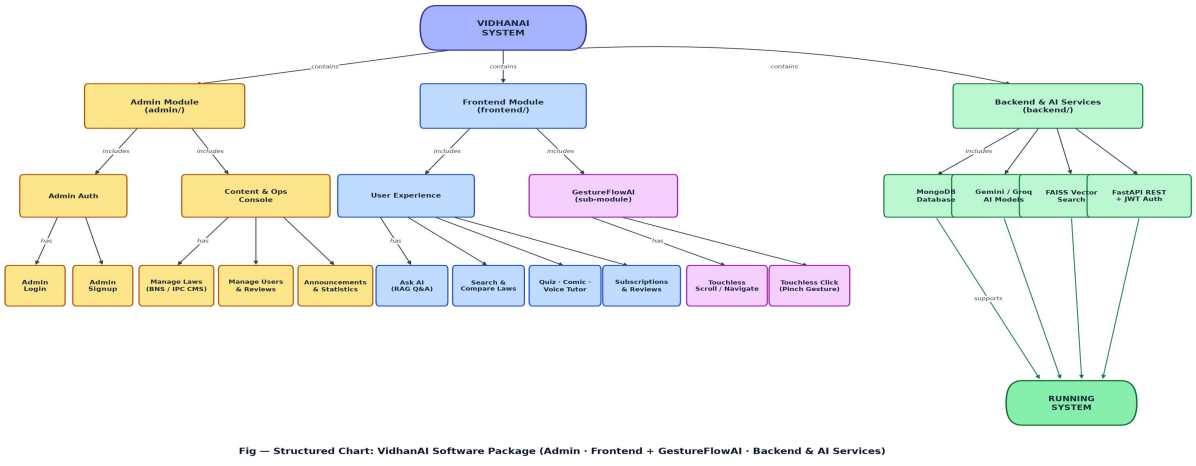


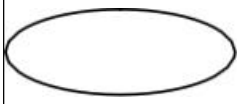
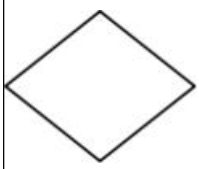
Fig — Structured Chart: VidhanAI Software Package (Admin · Frontend + GestureFlowAI · Backend & AI Services)

Figure 5.1 – Structured chart, VidhanAI module overview (Admin · Frontend + GestureFlowAI · Backend)

5.3 Activity Diagrams

Figures 5.2, 5.3 and 5.4 trace each of the three modules as its own control flow: the frontend's request/response cycle (including how it recovers from an expired session), the backend's full request lifecycle through auth, PlanGate and the AI services, and the admin console's authenticate-then-write cycle.

Basic Notation:

Symbol	Notation	Description
	Final Activity	Represents the end of the An activity diagram, also called as a final activity. It is shown by a bull's eye symbol.
	Initial Activity	Represents the starting point or first activity of the flow. It is denoted by a solid circle.
	Activity	Represents an activity. It is shown by a rectangle with rounded (almost oval) edges.
	Decision	Represents logic where a decision is to be made. It is shown by a diamond.
	Flow of Activity or Control	Shows the direction of the workflow in the activity diagram. It is depicted with an arrow.

Rounded start/end nodes mark where the flow begins and terminates (a filled black start node opens each diagram). Rectangles are actions, with a ruled line separating the action name

from an implementation note directly beneath it. Diamonds are decisions, and each outgoing arrow is labelled with the condition it represents. A side branch that terminates early — a rejected request, an expired session — ends at its own "end" node rather than rejoining the main flow; a curved arrow back into an upstream box shows a successful side-check (such as JWT verification) rejoining the sequence before it continues.

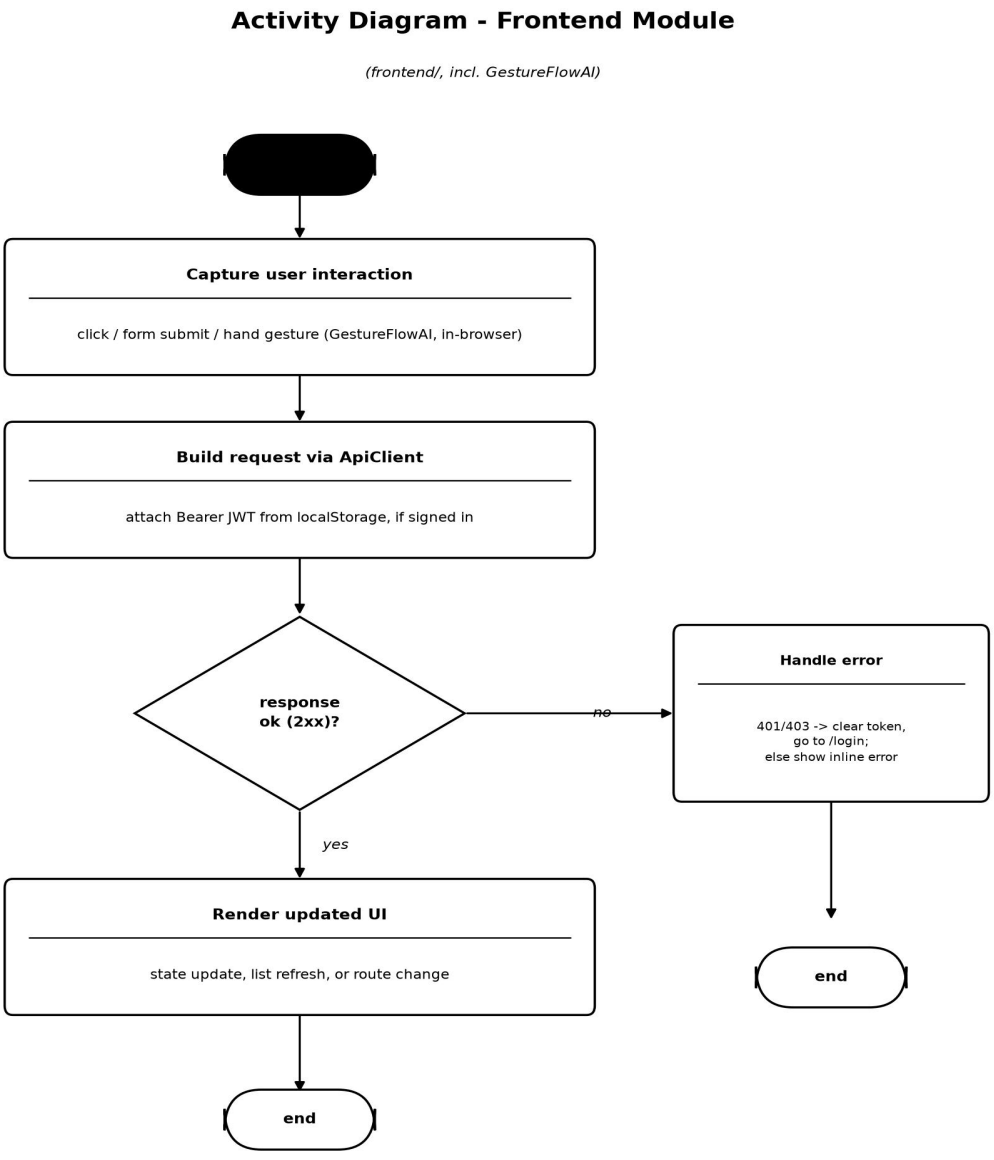


Figure 5.2 – Activity diagram, Frontend Module (frontend/, incl. GestureFlowAI)

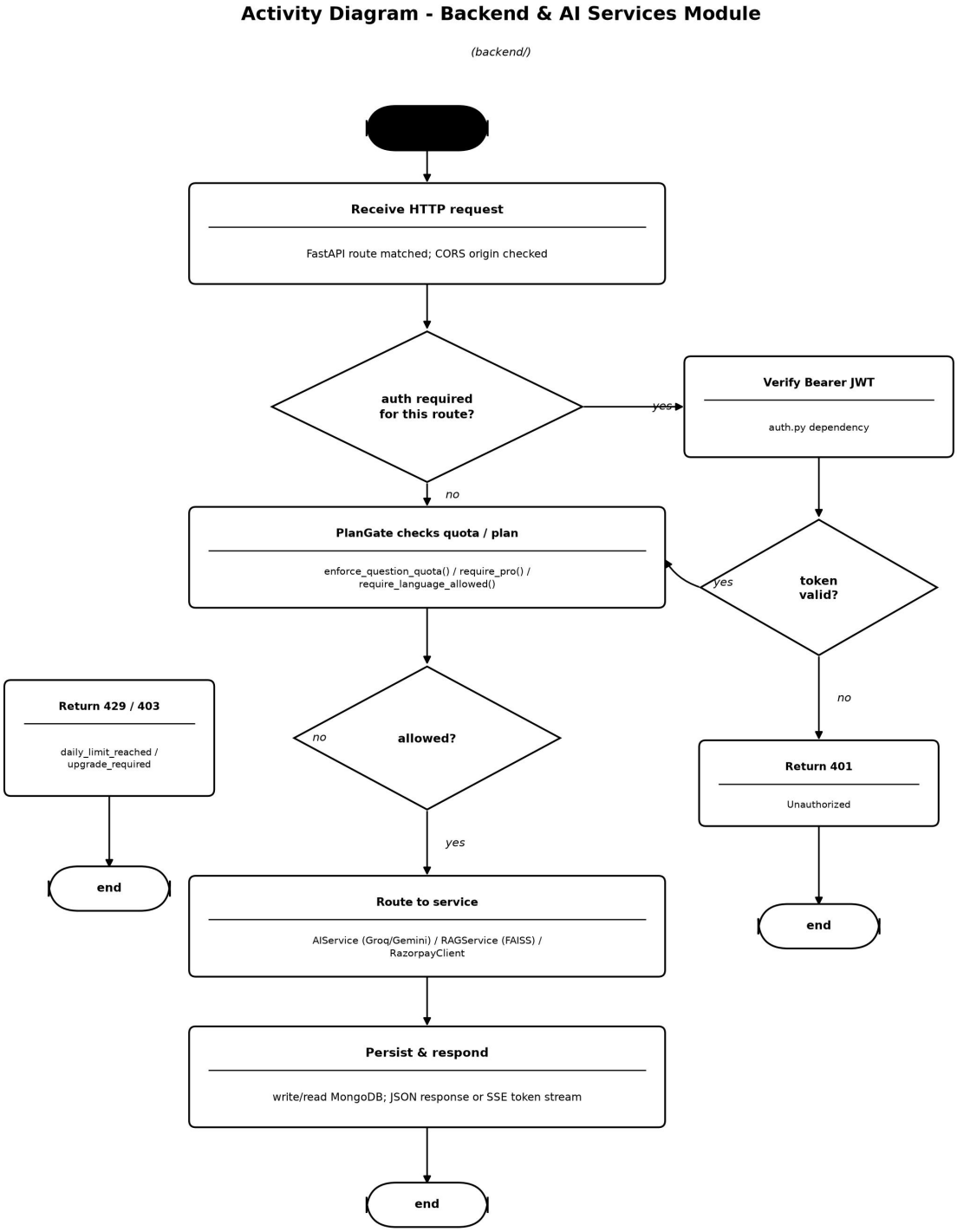


Figure 5.3 – Activity diagram, Backend & AI Services Module (backend/)

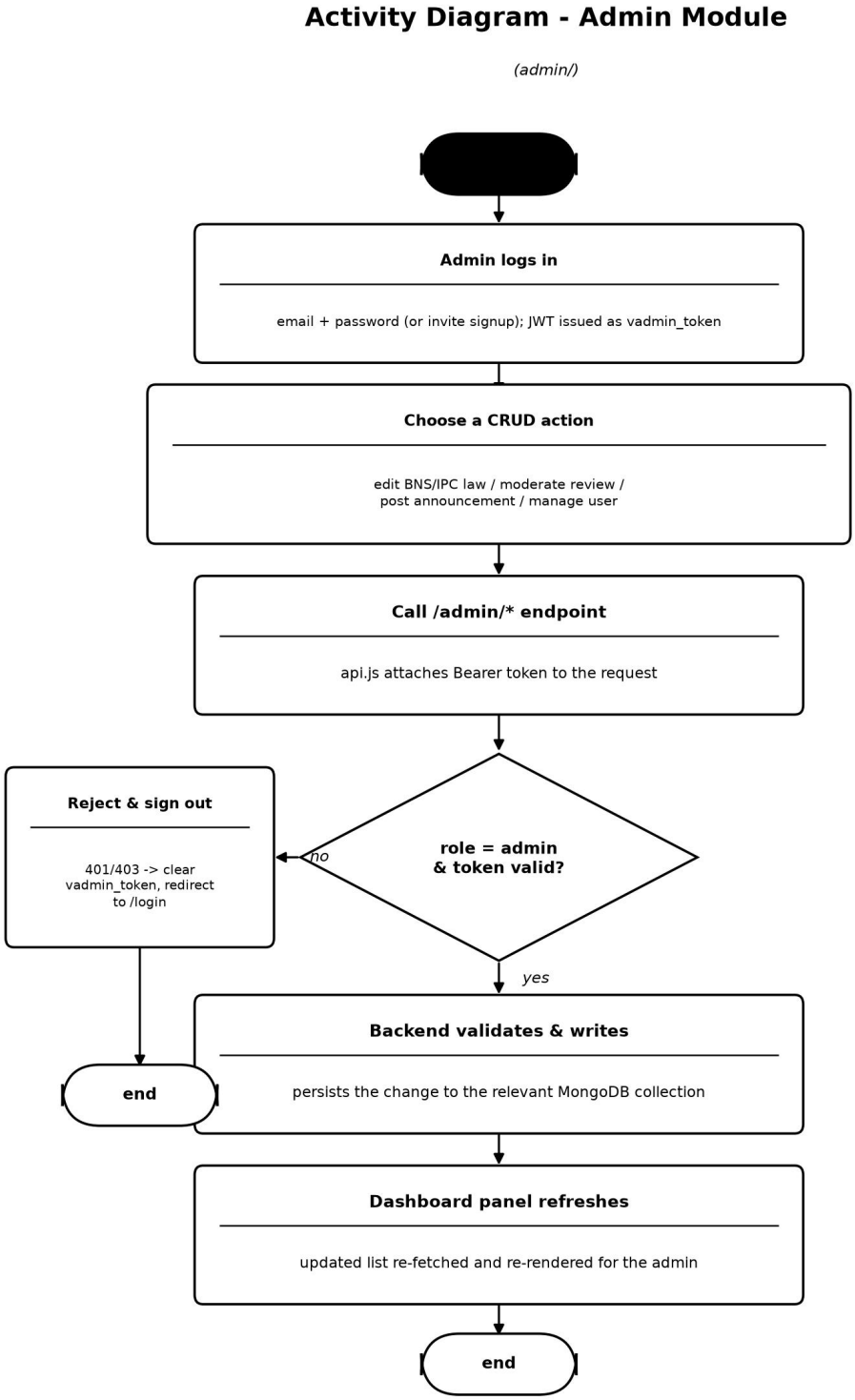
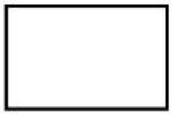
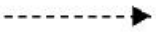


Figure 5.4 – Activity diagram, Admin Module (admin/)

5.4 Use Case Diagram

Figure 5.5 places every feature from Chapter 2 against the actor that can reach it. Anonymous visitors, Free users, Pro subscribers and the Administrator each reach a different slice of the system, and GestureFlowAI is drawn as its own nested boundary inside the frontend since it needs no login at all.

Basic Notations:

Symbol	Notation	Description
	System Boundary	Defines the scope of the system.
	Use Case	Represents a function provided by the system.
	Actor	Represents a user or external system.
	Association	Shows interaction between an actor and a use case.
	«include» Relationship	Indicates one use case always includes another.

Stick figures are actors — the roles interacting with the system (Anonymous Visitor, Free User, Pro Subscriber, Administrator, plus the external Razorpay and Google OAuth services). Ovals are use cases, grouped inside a rounded system-boundary box for each module — the frontend web app, its nested GestureFlowAI sub-boundary, and the Admin Console. A hollow-arrowhead line between two actors is a generalisation: Pro Subscriber inherits every use case reachable by Free User, which in turn inherits every use case reachable by Anonymous Visitor, so only the use cases each tier adds are drawn directly to it. A dashed arrow labelled «include» marks a use case that always triggers another as part of its own flow.

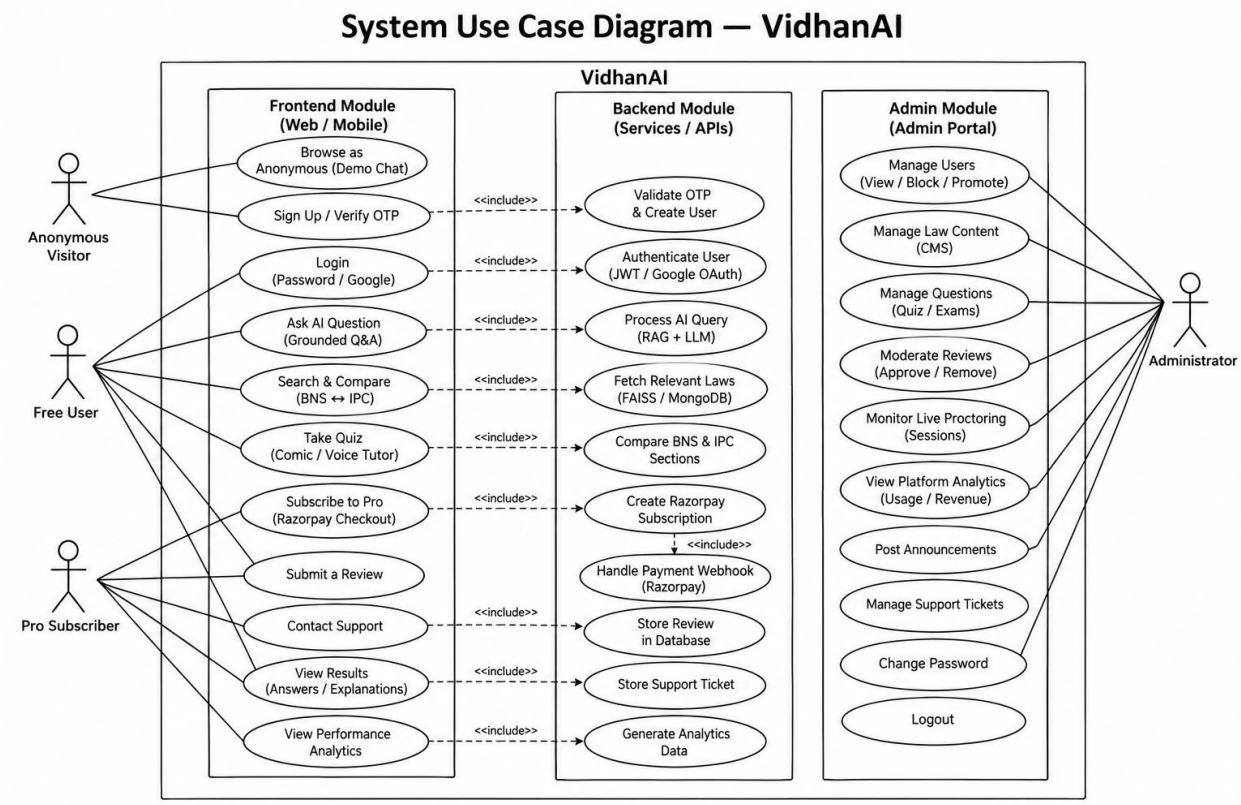


Figure 5.5 – System use case diagram

5.5 Class Diagram

Figure 5.6 separates the backend's domain classes (User, Subscription, Review, Query, and the LawSection hierarchy) from the service layer that operates on them, then places the Admin Console and Frontend — with GestureFlowAI nested inside it — as their own packages, connected to the backend only over REST.

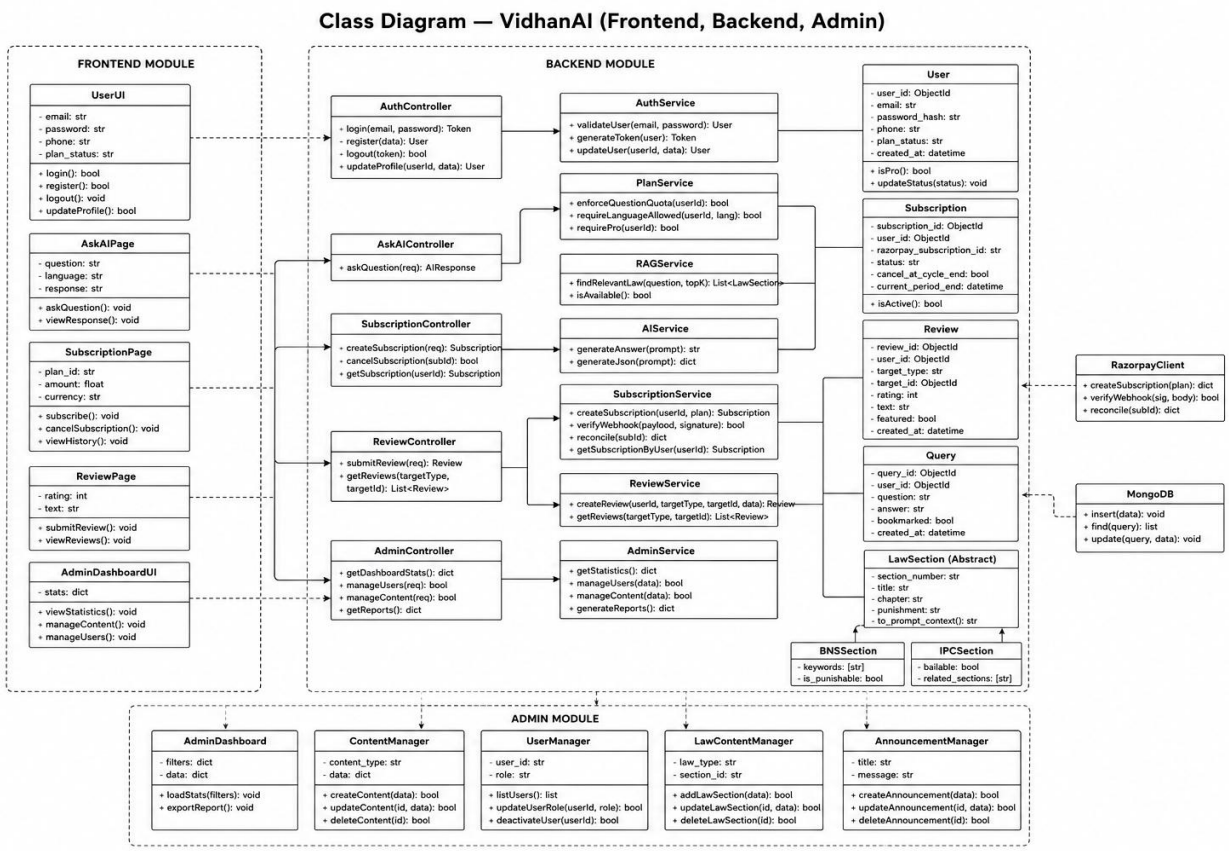


Figure 5.6 – Class Diagram of the VidhanAI System

5.6 Sequence Diagrams

Figure 5.7 shows the three modules as one message sequence, using the flow that exercises all of them together: the admin console authenticates and publishes an announcement, the backend persists it and queues the newsletter e-mail, and the citizen frontend later fetches and renders it — with each participant grouped under its owning module.

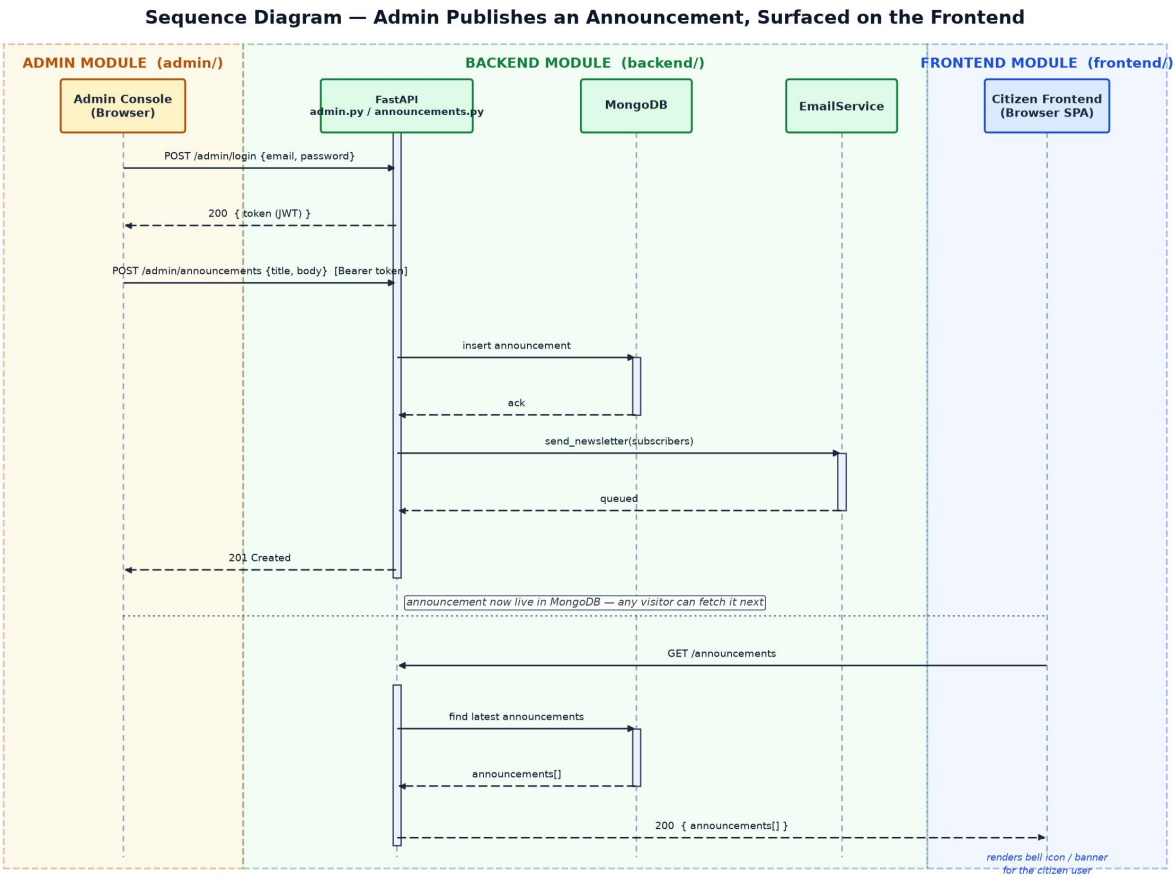


Figure 5.7 – Sequence diagram, admin announcement published & surfaced on frontend (Admin · Backend · Frontend)

5.7 Module Descriptions

5.7.1 Frontend Module (frontend/)

Inputs

User interaction events (clicks, taps, hand gestures via webcam); a JWT held in localStorage for signed-in requests; runtime config (VITE_API_URL) naming the backend origin.

Procedural Details

React Router switches between pages (Home, AskAI, QuizHub, ComicStory, Pricing, Reviews, …); a shared ApiClient wraps every fetch() call and attaches the Bearer token; AuthContext keeps the signed-in user in memory. GestureFlowAI (public/gesture-control.js)

runs a separate MediaPipe HandLandmarker loop that turns hand gestures into synthetic scroll/click events, entirely in-browser.

File / Data Interfaces

REST/JSON and Server-Sent Events to the backend only; the hand-landmark model is served as a static asset from public/models/; no direct database access.

Outputs

Rendered UI; outbound HTTP/SSE requests to backend/app; localStorage writes (auth token, gesture on/off state).

Implementation Aspects

Vite + React SPA. GestureFlowAI decouples its camera/AI loop from a requestAnimationFrame physics loop so gesture scrolling stays smooth independent of camera frame rate; requires the backend's CORS list to include the deployed frontend origin.

5.7.2 Admin Module (admin/)

Inputs

Admin credentials (or registration invite) at login/signup; a Bearer JWT held under vadmin_token; CRUD form submissions for laws, users, reviews and announcements.

Procedural Details

A second, independently deployed Vite + React app with its own login/signup screens and a Dashboard shell (Overview, UsersPanel, LawsPanel, ReviewsPanel, QueriesPanel, UpdatesPanel, SettingsPanel); every call goes through api.js, which attaches the Bearer token and forces a re-login on 401/403.

File / Data Interfaces

Calls the same backend under /admin/* (stats, users, laws, ipc-laws, reviews, announcements, admins); no direct database or filesystem access of its own.

Outputs

Moderated reviews, published announcements, edited BNS/IPC records, and user/admin deletions — all persisted through the backend into MongoDB.

Implementation Aspects

Deployed separately from the citizen frontend under its own origin, with an independent token key so an admin session and a citizen session never collide in the same browser.

5.7.3 Backend & AI Services (backend/)

Inputs

HTTP requests from both frontend applications (JSON bodies, Bearer JWTs); Razorpay webhooks with an HMAC-signed raw body.

Procedural Details

A single FastAPI app wires CORS, the routers (auth, quiz, tutor, comic, reviews, subscriptions, announcements, admin, …) and the services beneath them — AIService (Groq/Gemini), RAGService (FAISS retrieval), PlanGate (quota and plan enforcement) and RazorpayClient (billing). PlanGate is consulted before any AI call or Pro-only endpoint runs.

File / Data Interfaces

Reads vector/law_index.faiss and id_mapping.pkl for semantic search; reads and writes MongoDB collections (users, subscriptions, reviews, queries, announcements, …); calls the external Groq, Gemini and Razorpay APIs.

Outputs

JSON responses and SSE token streams to both frontends; MongoDB writes; outbound e-mails via EmailService; Razorpay subscription objects.

Implementation Aspects

Stateless request handling — all session state lives in the JWT or in MongoDB — so the service can scale horizontally; one backend instance serves admin/ and frontend/ alike under a shared, explicitly allow-listed CORS origin set.

CHAPTER 6

User Interface

6.1 Login

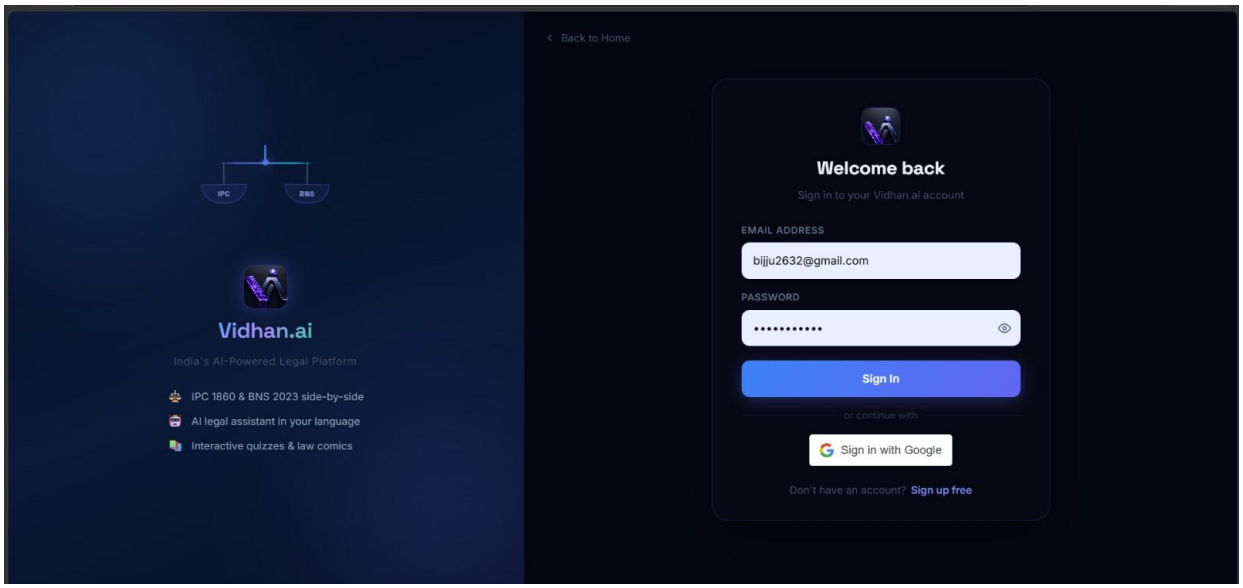


Figure 6.1 — Login

The login screen offers e-mail and password sign-in as well as “Continue with Google”. A new account then goes through a six-digit OTP screen, with a 10-minute expiry and a resend link. Once that succeeds, the JWT is stored on the client and the navbar switches over to the avatar menu.

6.2 Main Screen / Home Page

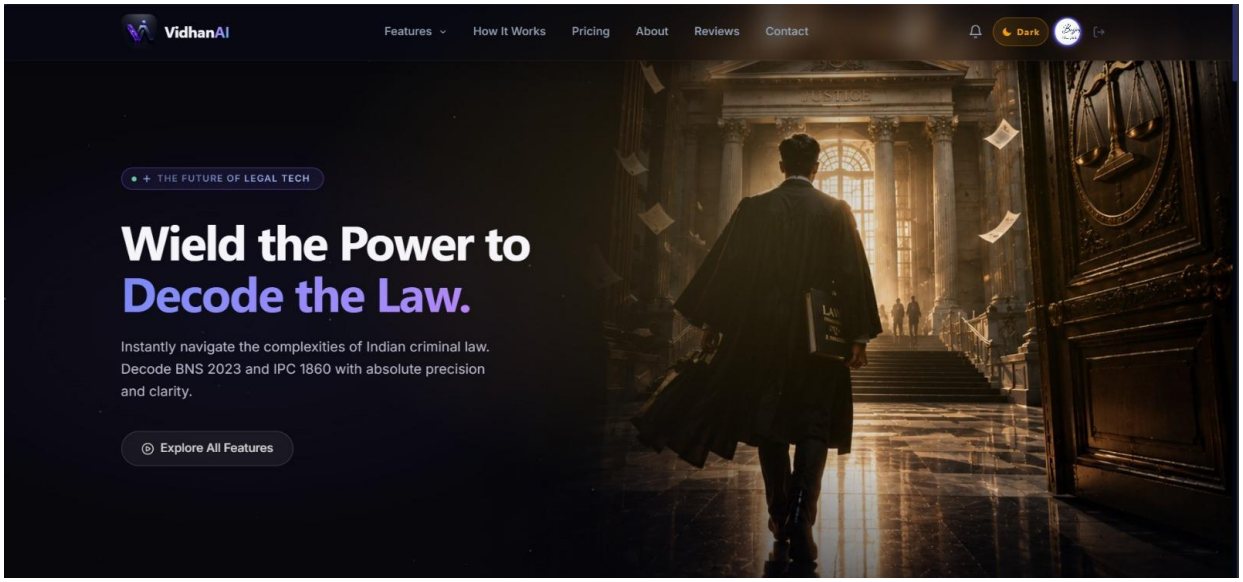


Figure 6.2 — Main Screen / Home Page

The landing page is deliberately cinematic: a hero with a 3D scene, a feature grid, the admin-curated “Real People, Real Justice” featured-review stack, Know-Your-Rights cards, and an anonymous demo chat capped at two questions per day per IP.

6.3 Menu

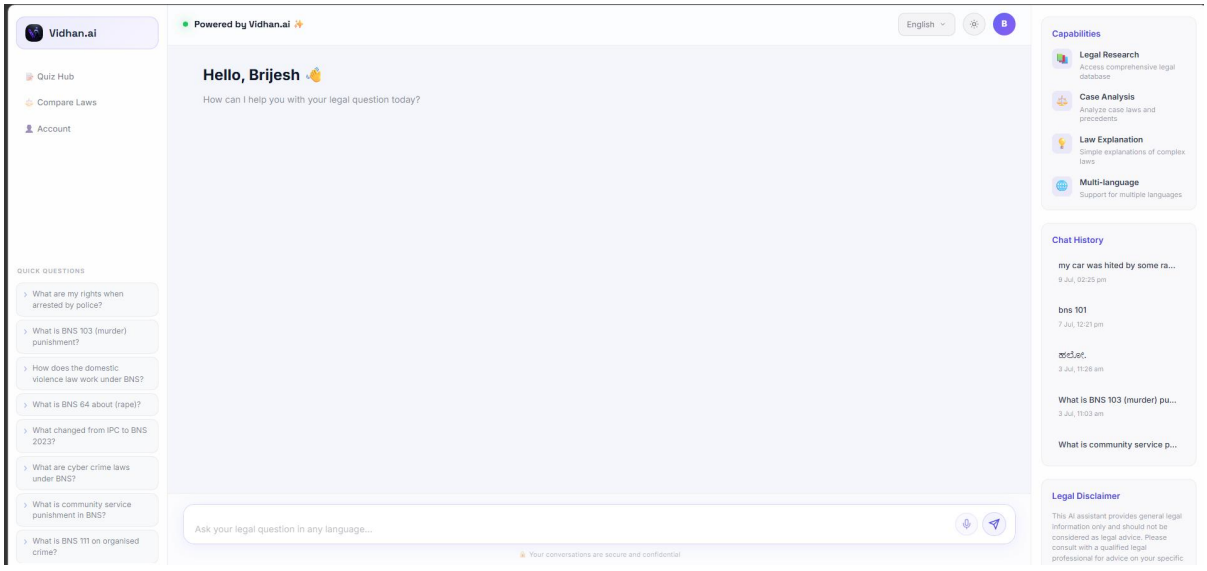


Figure 6.3.1 — Ask AI

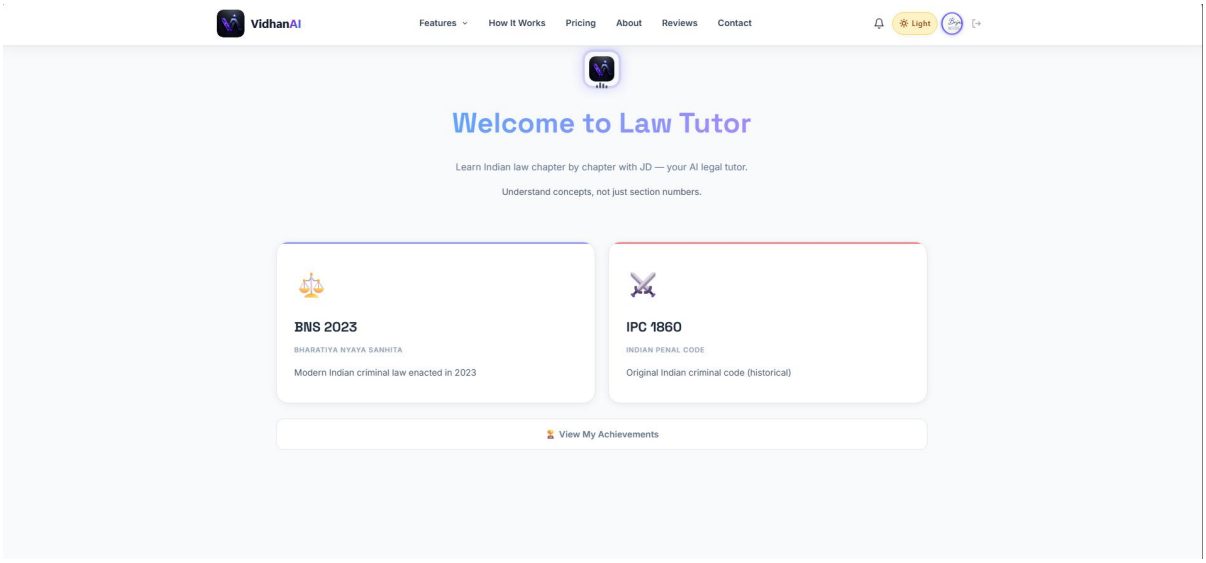


Figure 6.3.2 — AI Law Tutor

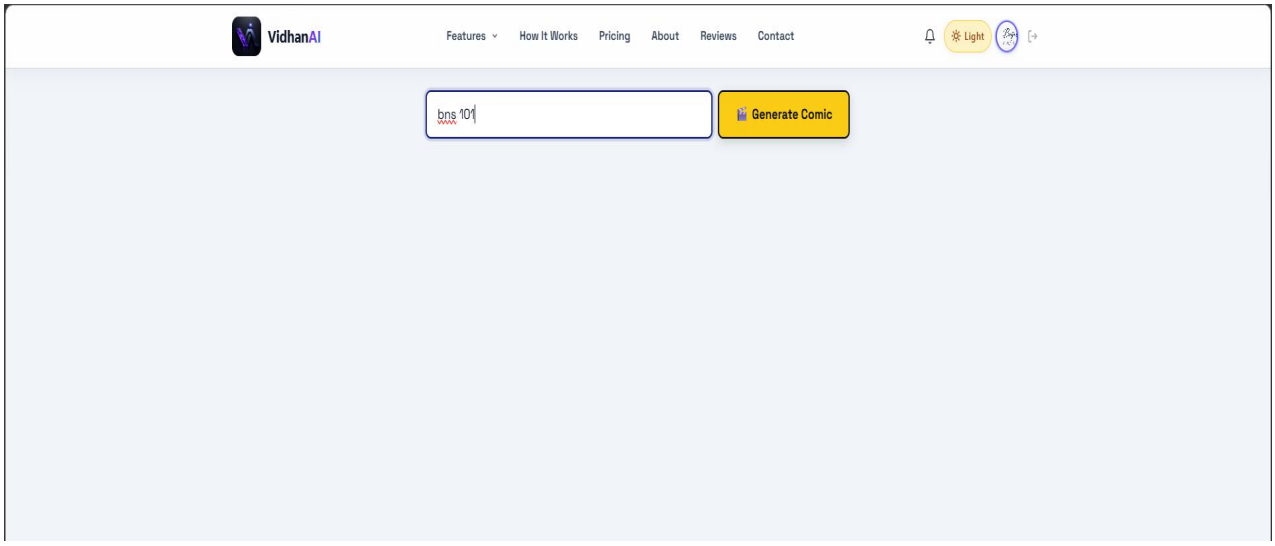


Figure 6.3.3 — Comic Story

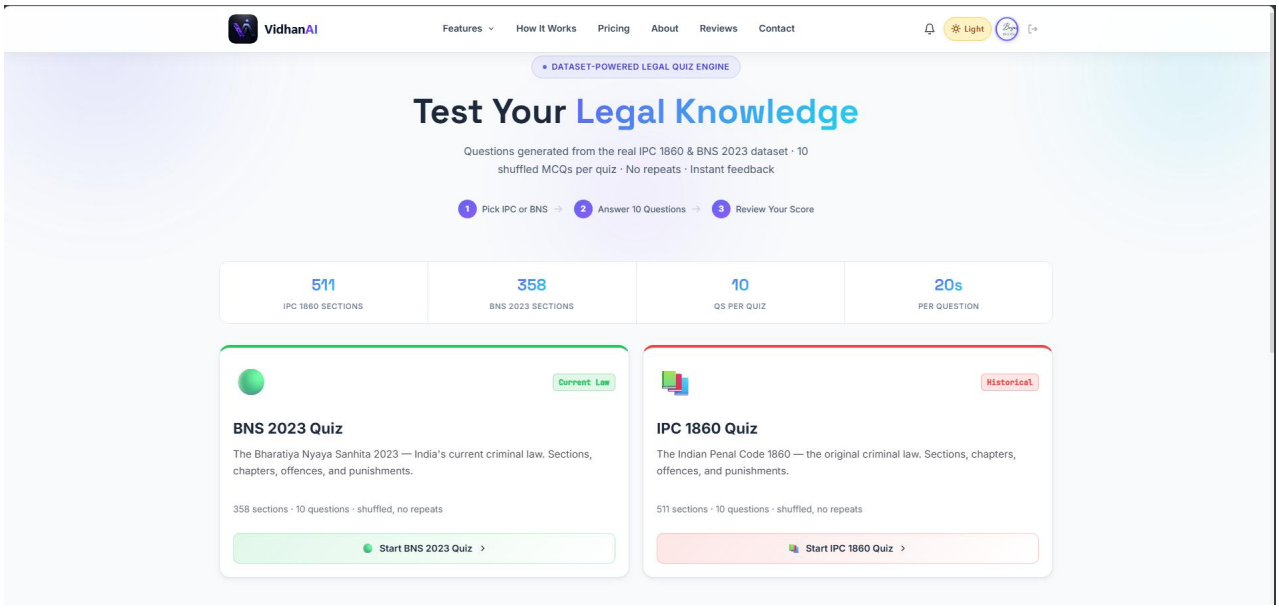


Figure 6.3.4 — Quiz Hub

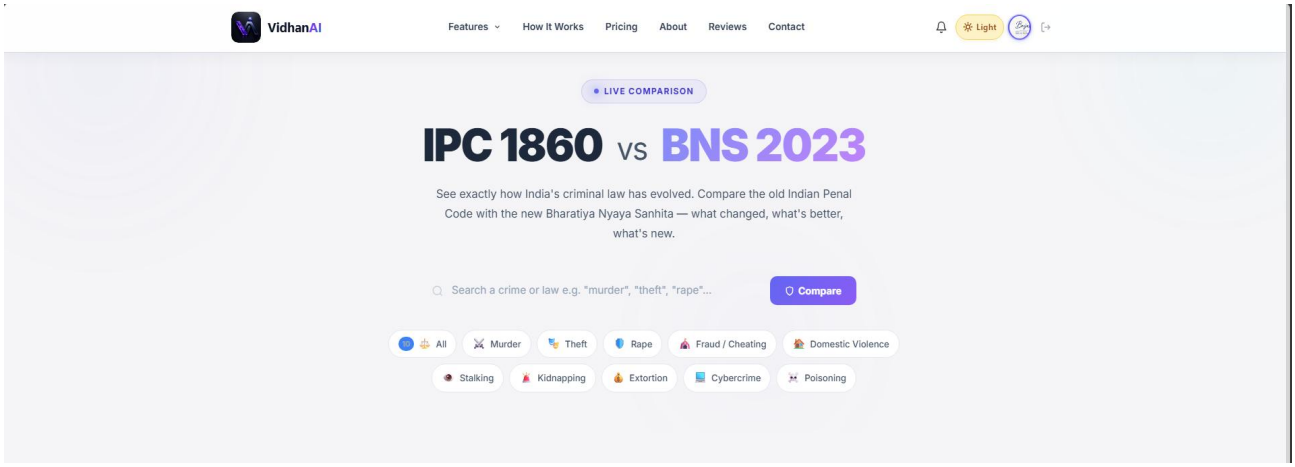


Figure 6.3.5 — Law Comparison (IPC vs BNS)

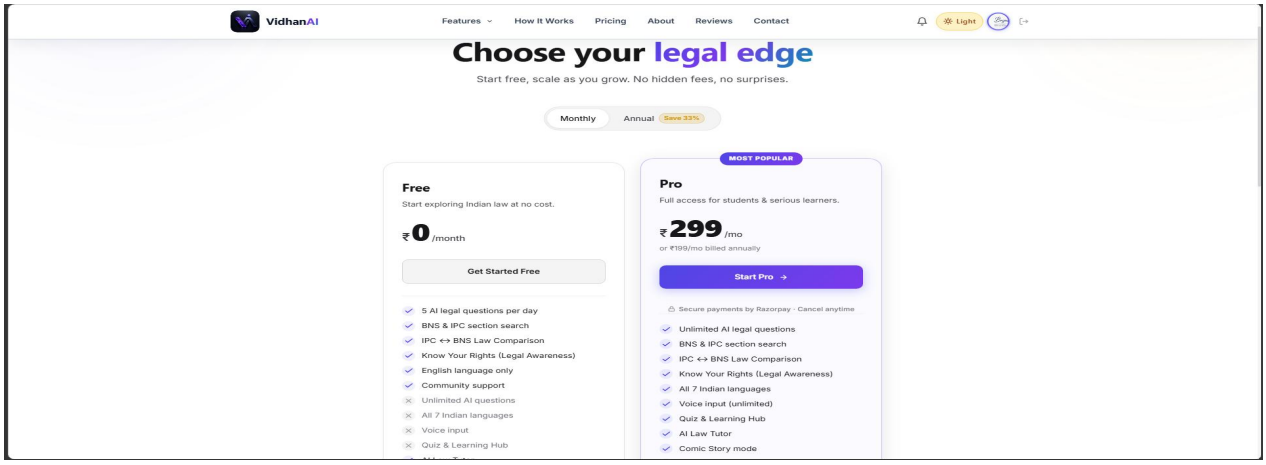


Figure 6.3.6 — Pricing

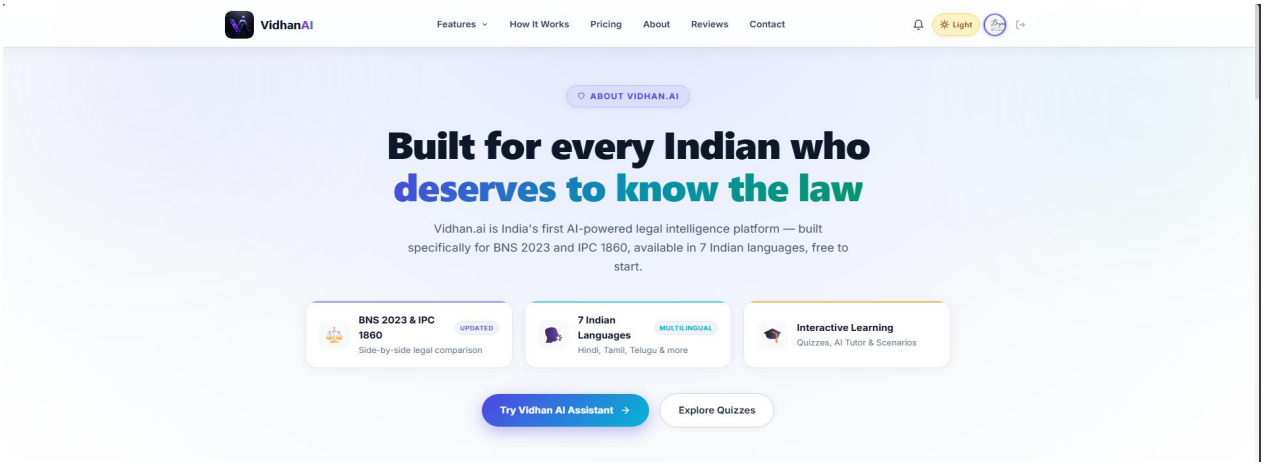


Figure 6.3.7 — About

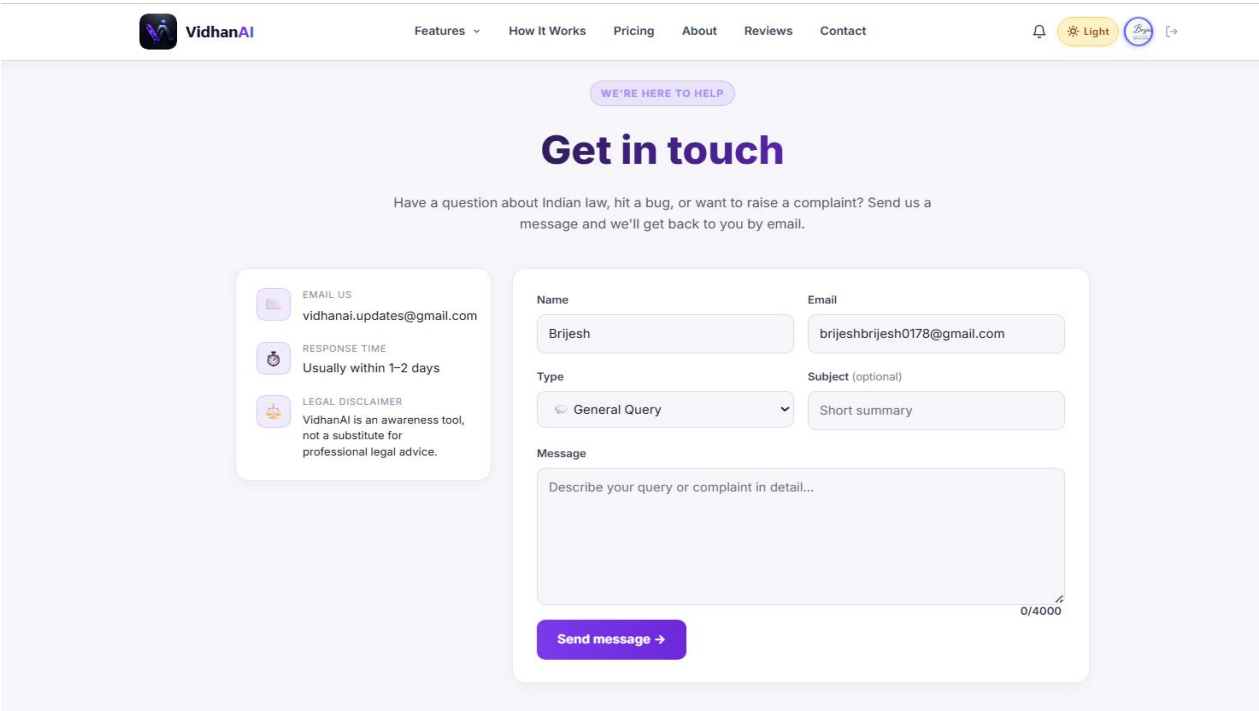


Figure 6.3.8 — Contact

The navbar carries Features (a dropdown of Ask AI, Comic Story, Tutor, Compare, and Quiz), How It Works, Pricing, About, Reviews, and Contact. On the right side are the notification bell, the theme toggle, the language selector, and the avatar, which shows a gold PRO badge for subscribers. The mobile hamburger menu repeats every one of these entries.

6.4 Data Store / Retrieval / Update

Nothing is stored locally: forms send JSON to the API and then re-render from whatever comes back. For example, profile edits (name, avatar, phone) update users, asking a question writes to history, and subscribing writes a subscription and then re-reads plan-status until it flips to Pro.

6.5 Validation

On the client, the app checks required fields, the e-mail pattern, password confirmation, review length, and the phone format. On the server, which is the real authority, it sanitises input, normalises phones to E.164 (defaulting to +91), enforces OTP expiry and quotas, and rejects a duplicate Pro phone — so every rule shown on the client is checked again on the server.

6.6 View

The read-only views include section detail pages (statute text, punishment, bailable/cognizable badges, and related sections), the IPC↔BNS comparison view, the plan status on the profile page (a chip with the renewal or “access until” date), and the chat history with bookmarks.

6.7 On-Screen Reports

For admins, the dashboard reports user counts, subscription states, query volumes, and the review queue. For citizens, the comparison summary acts as an on-screen generated report, and Q&A history entries can be exported as PDF.

6.8 Alerts

Non-blocking toasts confirm things like profile saves and review submissions. The subscription flow shows clear state banners (“Confirming your payment…”, “Welcome to Pro”, “Almost there…”). Running out of quota brings up an upgrade prompt that spells out

the Free limits, and cancelling asks for confirmation while stating plainly that the current period is not refunded.

6.9 Error Message

Error messages are specific and give the user a way forward. Trying to subscribe without a saved phone explains why and offers an “Add phone number” button that jumps to the profile. API failures show retry guidance instead of raw status codes, and the payment page never reports failure for a charge that is still processing.

6.10 GestureFlow AI

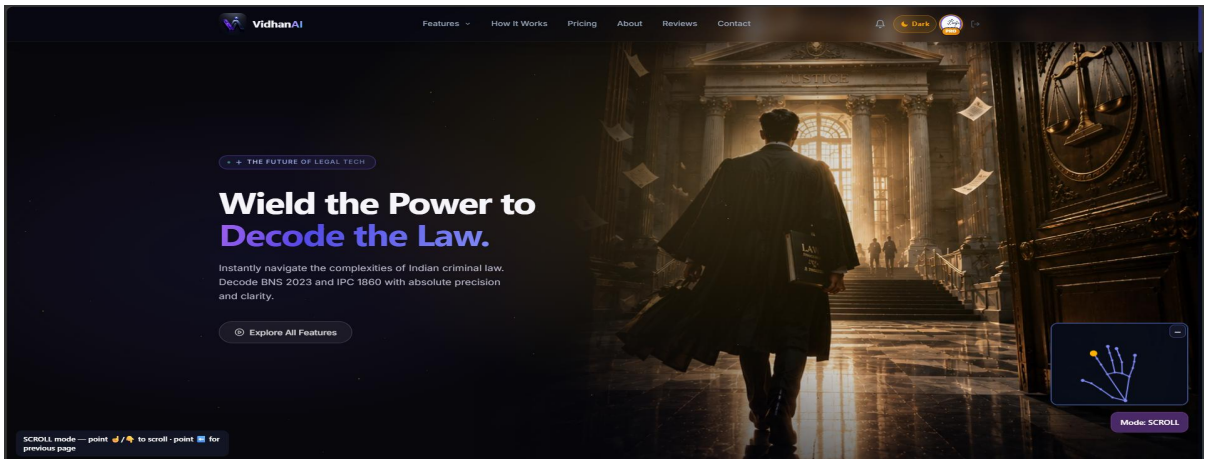


Figure 6.10 — GestureFlow AI

A small ✋ toggle sits fixed at the bottom-right of every page. Turning it on asks for camera permission and opens a dark hand-skeleton monitor panel — the 21 tracked hand points and bones are drawn live, but the camera video itself is never shown or sent anywhere. A status badge (bottom-left) narrates what the system currently sees, e.g. “ ✊ Scrolling UP — open your palm to stop”. A Mode button switches between SCROLL (point up/down to scroll, point left/right to go back/forward a page) and CURSOR (point to move a cursor, pinch to click — with a green outline previewing exactly what a pinch will hit). The same toggle, with a short gesture cheat-sheet and a link to the full guide, is repeated as a proper switch on the Profile page for anyone who prefers to turn it on from account settings rather than the floating button.

6.11 Admin Console

The admin console is a separate application with its own login, so none of the views below are reachable from a citizen session. Overview is the landing dashboard — cards for total users, reviews, and recent signups, colour-coded and trend-annotated. Users lists every account with a delete action. Reviews is the moderation queue: feature, unfeature, or delete any submitted review, with the featured ones being exactly what citizens see in the home-page “Real People, Real Justice” stack. Laws is the BNS/IPC content CMS — create, edit, and delete section entries in both codes. Queries is a searchable, paginated log of every AI exchange, filterable by user e-mail or keyword, useful for spotting weak answers without asking the user for a screenshot. Updates is where announcements are composed — a title, message, a tag (New Feature, Improvement, Bug Fix, Coming Soon), and an audience (all, Free, or Pro) — alongside a live newsletter-subscriber count. Settings covers the administrator’s own account: changing a password and registering additional admin accounts.

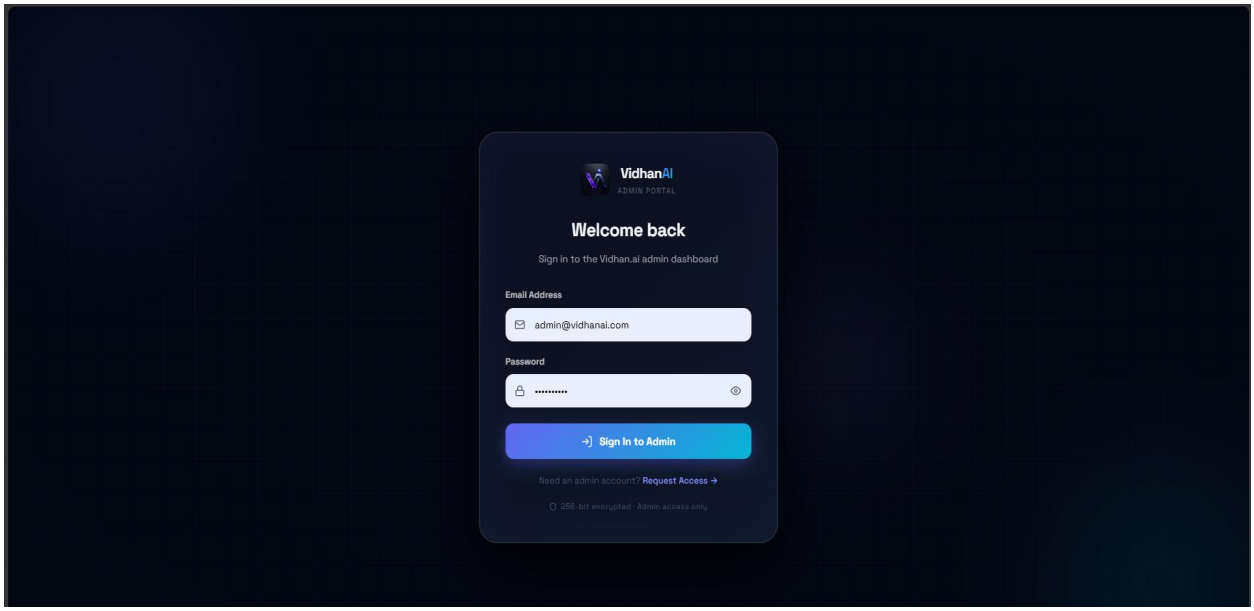


Figure 6.11.1 — Admin Login

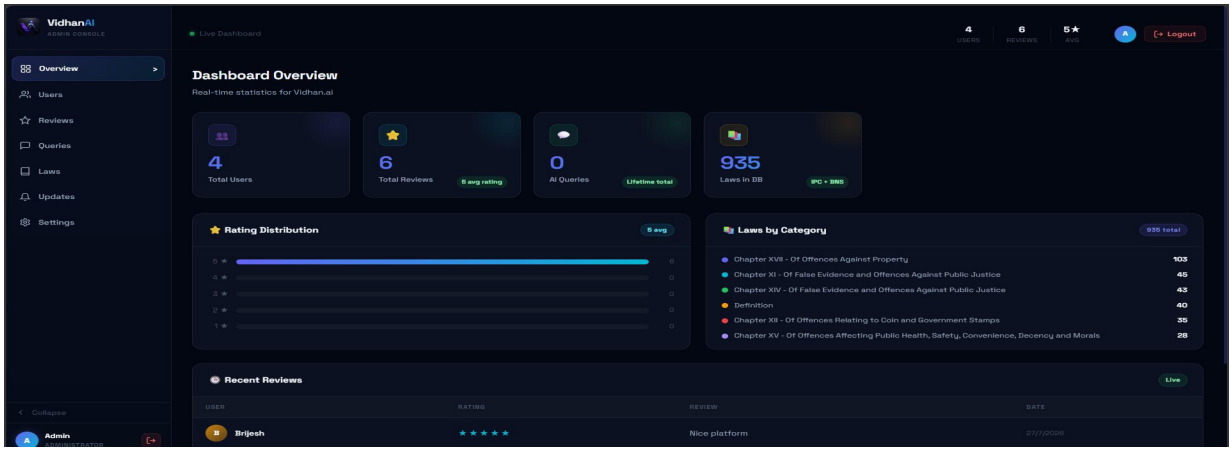


Figure 6.11.2 — Dashboard Overview

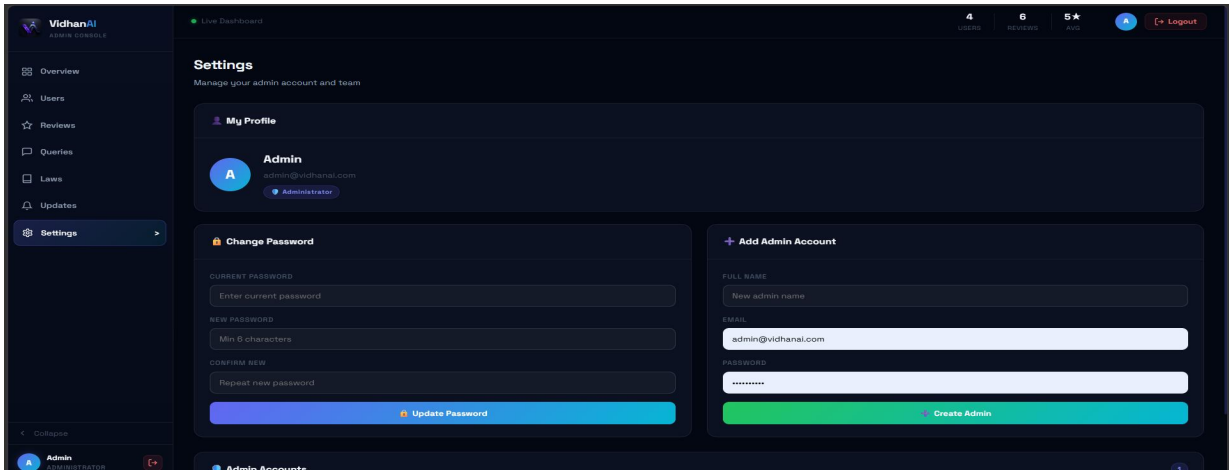


Figure 6.11.3 — Settings

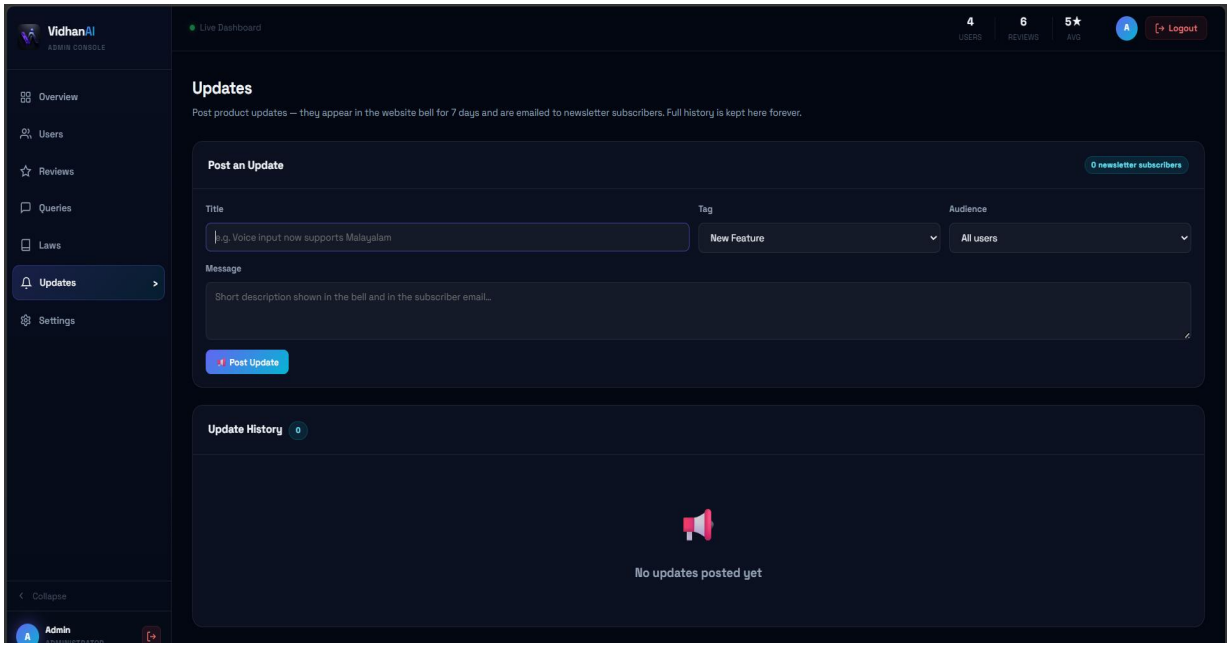


Figure 6.11.4 — Announcements

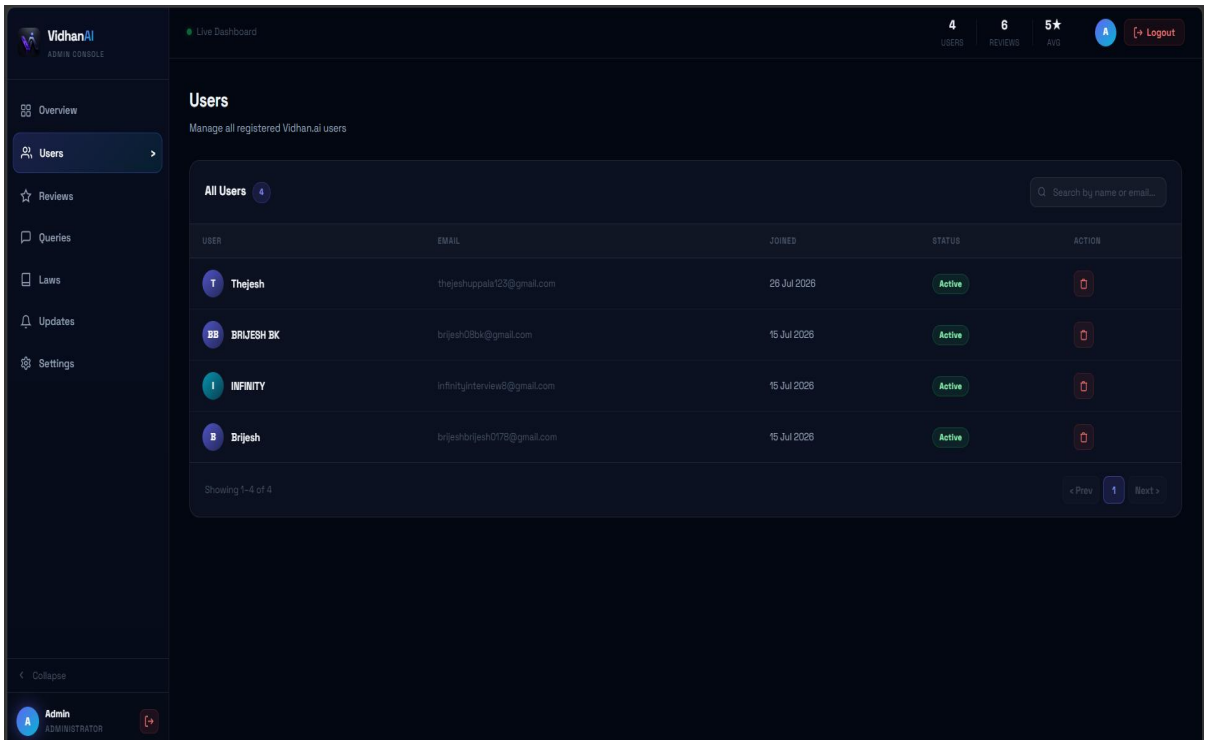


Figure 6.11.5 — User Management

CHAPTER 7

Testing

7.1 Introduction

Testing was done at three levels. Since most of the risk in this system lives in the integrations — payments, webhooks, and retrieval quality — the focus was on driving the real deployed endpoints, including the production backend on Render and the Razorpay test API, rather than mocking them. Several bugs were caught and fixed exactly this way.

7.2 Test Reports

Test ID	Test Case	Expected Result	Actual Result	Status
T-01	Subscribe without adding a phone number	System should display an error asking the user to add a phone number.	Error message displayed and subscription was not created.	PASS
T-02	Subscribe using a phone number already linked to another Pro account	System should prevent the subscription and display an appropriate message.	Subscription was blocked with a clear warning message.	PASS
T-03	Subscribe using a valid and unique phone number	System should create the subscription and automatically	Subscription was created successfully and user details were pre-filled.	PASS

		fill in user details.		
T-04	Send a webhook request with an invalid signature	System should reject the request without updating subscription details.	Invalid webhook request was rejected successfully.	PASS
T-05	Activate a valid subscription	User account should be upgraded to the Pro plan.	Subscription activated successfully and Pro features became available.	PASS
T-06	Check subscription when an old cancelled subscription exists	The active Pro subscription should remain unchanged.	Active subscription remained valid without any issues.	PASS
T-07	Cancel an active subscription	Subscription should remain active until the current billing period ends.	Cancellation was scheduled successfully and access remained available until expiry.	PASS
T-08	Validate different phone	Valid phone numbers	Phone numbers were validated	PASS

	number formats	should be accepted, invalid numbers rejected, and unauthorized users denied access.	correctly and invalid requests were rejected.	
T-09	Submit a user review	Review should be saved successfully and displayed after admin approval.	Review submission and moderation worked as expected.	PASS
T-10	Submit the contact form	Message should be delivered successfully and confirmation sent to the user.	Contact request was processed successfully.	PASS
T-11	Exceed the daily AI question limit for Free users	System should block additional questions and display an upgrade message.	Daily limit was enforced correctly and an upgrade message was displayed.	PASS

Eleven planned cases, all passing.

7.3 Testing Criteria

7.3.1 Unit Testing

Pure functions were tested on their own using the venv interpreter: input sanitisation; phone normalisation (E.164, +91 defaulting, length limits — the exact function listed in section 6.3.4); plan-gate quota arithmetic (the date rollover at IST midnight, the same $inc/reset pattern shown in section 6.3.1); and webhook signature construction (HMAC-SHA256 over the raw body, matching section 6.3.3).

7.3.2 Integration Testing

Each service boundary was tested against its real counterpart in test mode: Razorpay subscription create, fetch, and cancel; webhook delivery to the live Render URL with both correctly and incorrectly signed payloads; FAISS retrieval against the rebuilt index; Groq JSON-mode generation; Resend contact delivery; and MongoDB quota updates under repeated calls.

7.3.3 System Testing

Whole user journeys were run on the deployed site: signup → OTP → login; hitting the Free quota by asking questions; subscribe → checkout → PRO badge; cancel → "access until" state; and review submit → admin feature → appearance on the home page. The pass/fail outcomes of these journeys are the eleven cases recorded in the test report, section 8.2.

CHAPTER 8

Conclusion

VidhanAI shows that a grounded, honest legal-awareness assistant for Indian law can be built with today's open tooling. The choices that mattered most were grounding (retrieval picks the sections from the project's own corpus and the model only narrates them), enforcing every commercial rule on the server, and activating payments through two paths (webhook plus API reconciliation) so that money and access can never stay out of step. The system runs in production across Vercel, Render, and MongoDB Atlas, goes through a full subscription lifecycle in Razorpay's test environment, and — as Chapter 8 records — had its most important flows verified against the live deployment, not just on a development machine.

The project also has known limits, and being upfront about them is part of the same honesty principle that shapes the product itself:

- Retrieval quality is limited by the embedding model. all-MiniLM-L6-v2 occasionally misses the right section for an awkwardly phrased question and returns a nearby but wrong one even though the correct section is in the corpus.
- The free-tier production host (512 MB) cannot hold the embedding model, so the live site runs keyword retrieval unless DISABLE_VECTOR_SEARCH is turned off on a larger instance.
- Payments are test-mode only. No real money can move until live Razorpay keys (KYC) are set up, and the checkout window itself shows Razorpay's own test-mode indicators.
- Renewal and cancellation events that happen months later still depend on webhook delivery. Reconciliation covers the initial purchase window, but no one is watching the screen for these later lifecycle events.
- Answer quality in non-English languages rests on Gemini's Indic abilities and has not yet been evaluated systematically.
- One older endpoint (the profile-picture update) still authenticates using the e-mail in the request body instead of the JWT, and should be hardened to match the pattern used by the phone endpoints.

Chapter 9

Future Scope

- Improve retrieval with a stronger embedding model, or a cross-encoder re-ranker over the top-20 results, to close that class of miss; and index richer per-section text at build time.
- Go live commercially: complete Razorpay KYC, switch on live keys behind the existing test-key guard, and add GST-compliant invoicing.
- Widen the corpus by adding BNSS 2023 (procedure) and BSA 2023 (evidence) as first-class datasets that use the same grounding pipeline.
- Build native mobile apps on top of the existing API, with offline section reading.
- Add an evaluation harness — a labelled query→section test set run in CI — so retrieval regressions get caught automatically.
- Extend admin analytics with cohort retention, question-topic clustering, and revenue dashboards.

Abbreviations and Acronyms

Term	Expansion
BNS	Bharatiya Nyaya Sanhita, 2023
IPC	Indian Penal Code, 1860
BNSS / BSA	Bharatiya Nagarik Suraksha Sanhita / Bharatiya Sakshya Adhiniyam, 2023
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
FAISS	Facebook AI Similarity Search
JWT	JSON Web Token
OTP	One-Time Password
SPA	Single-Page Application
API / REST	Application Programming Interface / Representational State Transfer
CFD / DFD	Context Flow Diagram / Data Flow Diagram
ER	Entity-Relationship
UML	Unified Modeling Language
HMAC	Hash-based Message Authentication Code
TTS	Text-To-Speech
UPI	Unified Payments Interface
CDN	Content Delivery Network
KYC	Know Your Customer
PCI	Payment Card Industry (data security standard)

CHAPTER 10

Bibliography

- The Bharatiya Nyaya Sanhita, 2023 (Act No. 45 of 2023). Ministry of Law and Justice, Government of India.
- The Indian Penal Code, 1860 (Act No. 45 of 1860). Ministry of Law and Justice, Government of India.
- Ramírez, S. *FastAPI Documentation.*
- Meta Platforms, Inc. *React Documentation.*
- You, E. *Vite Documentation.*
- MongoDB Inc. *MongoDB Manual.*
- Johnson, J., Douze, M., & Jégou, H. (2021). *Billion-scale Similarity Search with GPUs.* IEEE Transactions on Big Data.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* Advances in Neural Information Processing Systems (NeurIPS).
- Groq Inc. *Groq API Documentation.*
- Google AI. *Gemini API Documentation.*
- Razorpay Software Private Limited. *Razorpay API and Subscription Documentation.*
- Resend Inc. *Resend Email API Documentation.*
- Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT).* Internet Engineering Task Force (IETF).
- Google Developers. *OAuth 2.0 Authentication Documentation.*.